

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	vi
Elenco delle tabelle	vii
Elenco dei codici	viii
1 Introduzione	1
1.1 Motivazione e Contesto	1
1.2 Definizione del Problema e Gap nella Ricerca	2
1.3 Obiettivi della Ricerca	3
1.4 Organizzazione della Tesi	4
2 Background e Stato dell'Arte	5
3 Background e Stato dell'Arte	6
3.1 Paradigmi delle Architetture API Moderne	6
3.1.1 Evoluzione dei Sistemi Distribuiti	6
3.1.2 Tassonomia degli Stili Architettureali e di Comunicazione	7
3.1.3 Il Fenomeno dell'API Sprawl	8
3.2 Il Pattern dell'API Gateway in Ottica Zero-Trust	9
3.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete	9
3.2.2 Criteri di Selezione e Giustificazione Architettureale	10
3.2.3 Enforcing del Paradigma Zero-Trust	10
3.3 Tassonomia delle Minacce e Vulnerabilità Logiche	11
3.3.1 Il Semantic Gap e l'Evoluzione del Panorama delle Minacce	11
3.3.2 Categorie di Fallimento Architettureale	12
3.3.3 Dal Modello Teorico alla Necessità di Valutazione Strutturata	13
3.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti	13
3.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST	13
3.4.2 Asimmetria Informativa e il Box Gradient	14
3.4.3 Contract-Driven Security e Approcci Ibridi	15

4	Metodologia, Architettura e Principi di Design	17
4.1	Principi Fondamentali: il Perché delle Scelte Architettureali	17
4.1.1	Agnosticismo Applicativo (D1.P1)	18
4.1.2	La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)	19
4.1.3	Config-Driven Development (D3.P1)	20
4.1.4	Riproducibilità Deterministica (D3.P4)	21
4.2	Architettura dell'Assessment Engine	21
4.2.1	Flusso di Dipendenze Monodirezionale (D1.P2)	22
4.2.2	Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	22
4.2.3	Split State e Immutabilità (D1.P3)	23
4.2.4	Gateway-Agnostic Adapter (D1.P6)	25
4.2.5	Streaming Evidence Store (D4.P1)	25
4.2.6	Assenza di Dipendenze di Stato Esterno (D3.P5)	26
4.3	Domini di Sicurezza e Box Gradient Applicato	26
4.3.1	Tassonomia degli Otto Domini di Assurance	27
4.3.2	Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	28
5	Implementazione	30
5.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	30
5.1.1	Fail-Safe Error Isolation (D4.P3)	34
5.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	34
5.1.3	Costruzione dei Contesti di Esecuzione (D1.P3)	36
5.2	Discovery Dinamico e Risoluzione del Grafo	36
5.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	37
5.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	38
5.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	39
5.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	39
5.3.2	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	40
5.3.3	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	41
5.3.4	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	42
5.3.5	Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)	42
5.3.6	Path Seed System (D2.P7)	44
5.3.7	Catalogo dei Payload di Attacco (D2.P8)	44
5.3.8	Teardown Best-Effort in Ordine LIFO (D4.P6)	45
5.4	Gerarchia dei Connettori e Integrazione Tool Esterni	45
5.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	46
5.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	47
5.4.3	Classificazione A/B dei Connector (D2.P6)	48
5.4.4	Isolamento della Dipendenza AGPL (D7.P2)	48
5.5	Implementazione del Gateway Adapter	48
5.5.1	<code>KongGatewayAdapter</code> : Accesso Read-Only all'Admin API	49

5.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	49
5.6	Evidence Store e Sanitizzazione delle Credenziali	50
5.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	51
5.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	51
5.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	53
6	Validazione sperimentale e risultati	55
6.1	Topologia del Testbed Sperimentale	55
6.1.1	Architettura del Testbed	56
6.1.2	Configurazione Intenzionale per la Copertura dei Finding	57
6.1.3	Struttura RBAC e Provisioning degli Utenti	58
6.2	Release-Readiness e Integrità Architetturale	58
6.2.1	Analisi Statica del Codebase	59
6.2.2	Dual-Layer Type Safety (D6.P3)	59
6.2.3	Indicatori di Qualità dell'Ingegneria di Produzione	60
6.3	Copertura Funzionale e Idempotenza dell'Assessment	61
6.3.1	Analisi dei Risultati per Dominio	62
6.3.2	Idempotenza e Validazione del Teardown	63
6.4	Footprint Computazionale e Benchmarking del DAG	64
6.4.1	Regime di Esecuzione: IO-Bound, non CPU-Bound	64
6.4.2	Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione	65
	Sintesi della Validazione	66
7	Discussione e Sviluppi Futuri	68
7.1	Analisi Critica e Limiti del Paradigma Contract-Driven	68
7.1.1	Il Paradosso del Ground Truth (D3.P2)	69
7.1.2	La Tensione dell'Astrazione (D1.P6)	69
7.1.3	La Dipendenza dall'Admin API (D4.P5)	70
7.2	Applicabilità Industriale e Integrazione DevSecOps	71
7.2.1	Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)	71
7.2.2	Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)	72
7.2.3	Variabilità di Privilegio nei Deployment Industriali	73
7.3	Sviluppi Futuri	73
7.3.1	Estensioni Operative	74
7.3.2	Traiettorie di Ricerca Aperta	75
8	Conclusioni	77
8.1	Sintesi del Lavoro e Risoluzione del Problema Iniziale	77
8.2	Contributi Scientifici e Ingegneristici	78
8.3	Considerazioni Finali sull'Assurance Continuo	79
	Bibliografia	82

Elenco delle figure

Figura 5.1 Layout di <code>src/</code> con mapping alle fasi della pipeline.	32
--	----

Elenco delle tabelle

Tabella 4.1	Proprietà APIGuard Assurance per dominio.	18
Tabella 4.2	Domini di Assurance di APIGuard Assurance.	27
Tabella 4.3	Box Gradient applicato in APIGuard Assurance (D3.P3).	29
Tabella 5.1	Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).	31
Tabella 5.2	Gerarchia delle eccezioni custom (Fasi 1–4 bloccanti; Fasi 5–6 recupe- rate localmente).	35
Tabella 6.1	Componenti del testbed e versioni al Milestone 1.	56
Tabella 6.2	Analisi statica della codebase APIGuard Assurance v0.1.0	59
Tabella 6.3	Verdetto release-readiness al Milestone 1.	61
Tabella 6.4	KPI di idempotenza su due run indipendenti.	61
Tabella 6.5	Status e finding count per test.	62
Tabella 6.6	Verifica live del teardown post-run.	63
Tabella 6.7	Metriche di sistema sui due run.	64
Tabella 6.8	Topologia del Directed Acyclic Graph (DAG) a Milestone 1.	65

Elenco dei codici

Capitolo 1

Introduzione

Il confine tra ciò che un'architettura a microservizi espone intenzionalmente e ciò che diventa raggiungibile per effetto della loro composizione è raramente definito con precisione: ogni servizio aggiunge interfacce programmabili al perimetro, e il numero di vettori di attacco potenziali cresce con le interazioni tra servizi, non con il numero di servizi in sé. Gli strumenti concepiti per il security testing delle applicazioni tradizionali operano sulla sintassi delle richieste HyperText Transfer Protocol (HTTP): l'assenza di una rappresentazione formale del contratto che quelle richieste dovrebbero rispettare è strutturale, e ne preclude l'efficacia prima ancora di raggiungere la logica applicativa su cui le vulnerabilità semantiche si innestano. Questo lavoro colma quel gap progettando e validando un framework di security assessment che deriva la propria conoscenza del target dalla specifica formale dell'Application Programming Interface (API) e la usa per costruire verifiche riproducibili, priorizzate e tracciabili su qualsiasi sistema REST documentato.

1.1 Motivazione e Contesto

L'adozione delle architetture a microservizi cloud-native ha redistribuito la complessità applicativa su un numero crescente di servizi autonomi, ciascuno responsabile di un sottodominio del sistema e comunicante con gli altri tramite interfacce programmabili. La conseguenza diretta è la moltiplicazione della superficie esposta: ogni servizio aggiunge endpoint al perimetro, spesso in assenza di un inventario centralizzato e di una governance sistematica del ciclo di vita delle API. Il fenomeno genera tre categorie di esposizione non intenzionale: le API deprecated che continuano a rispondere per mancata disattivazione, gli endpoint raggiungibili ma assenti da qualsiasi specifica, e le interfacce amministrative esposte per errore di configurazione. Tutte e tre sono invisibili agli strumenti che non dispongono di una rappresentazione formale della superficie esposta: uno scanner che costruisce il proprio modello dal traffico osservato o dalla documentazione

disponibile non può rilevare ciò che non è documentato o non ha ancora prodotto traffico. Il Capitolo 3 ne analizza le implicazioni in dettaglio.

L'impatto di questa esposizione sul panorama della sicurezza è documentato da più fonti convergenti. L'Open Web Application Security Project (OWASP) API Security Top 10 [?] formalizza le dieci categorie di rischio più frequenti nelle API REST, con le prime cinque interamente composte da vulnerabilità semantiche: violazioni di autorizzazione a livello di oggetto (Broken Object Level Authorization (BOLA)), difetti di autenticazione, esposizione non intenzionale di proprietà e consumo incontrollato di risorse. Le API hanno consolidato il proprio ruolo di vettore primario negli incidenti di sicurezza enterprise [?], con una frequenza di violazioni correlata a configurazioni errate o a controlli di autorizzazione insufficienti che segnala un disallineamento strutturale tra la velocità con cui le API vengono esposte e la maturità degli strumenti disponibili per valutarne la sicurezza in modo sistematico.

La combinazione di scala e complessità semantica rende il security assessment manuale inadeguato come pratica sistematica: un penetration test condotto su un campione ristretto di endpoint, in un momento specifico, produce un'osservazione puntuale che non è confrontabile con esecuzioni successive e non scala all'intera superficie esposta. La necessità è dotare il processo di assessment di proprietà formali: riproducibilità deterministica dei risultati, tracciabilità dei verdeti alla specifica contrattuale del target, e integrabilità nelle pipeline di sviluppo continuativo senza richiedere intervento manuale a ogni release. Le ragioni per cui gli strumenti esistenti non soddisfano congiuntamente questi requisiti sono analizzate nella Sezione 1.2.

1.2 Definizione del Problema e Gap nella Ricerca

Gli approcci esistenti al security assessment delle API REST presentano tre limitazioni strutturali distinte, ciascuna radicata in una scelta di design diversa e resistente a soluzioni additive. Identificarle separatamente è necessario perché richiedono risposte architetturali diverse: un framework che ne risolva solo una o due produce un assessment che rimane parzialmente cieco, o non portabile, o non ripetibile.

Il primo gap riguarda la capacità di osservazione. Gli scanner Dynamic Application Security Testing (DAST) e i fuzzer generici operano sulla sintassi delle richieste HTTP: generano payload, osservano i codici di risposta e cercano pattern anomali nel traffico. Questa strategia è efficace per le vulnerabilità che si manifestano nella forma della request o della response, come le injection sui parametri. Le vulnerabilità semantiche delle API REST appartengono a una classe diversa: si manifestano come richieste sintatticamente corrette che violano le assunzioni implicite del sistema sul chiamante. Una violazione di autorizzazione orizzontale (BOLA) è, dal punto di vista dello scanner, una richiesta GET

perfettamente formata verso un endpoint documentato; il problema è che il chiamante accede a una risorsa su cui i propri privilegi sono insufficienti. Un bypass della revoca dei token è una richiesta autenticata con un token che il sistema avrebbe dovuto invalidare; lo scanner, privo di conoscenza del contratto che ne governa il ciclo di vita, produce una risposta `2xx` e la classifica come esito corretto. Il combinatorial wall, discusso nel Capitolo 3, è la conseguenza diretta di questa cecità: i payload che raggiungono la logica semantica dell'API sono quelli costruiti nel rispetto del contratto, e uno scanner privo di quella conoscenza non può costruirli sistematicamente.

Il secondo gap riguarda la portabilità. Gli strumenti specializzati di security assessment per API si distribuiscono agli estremi di uno spettro: da un lato gli strumenti vendor-specific, progettati per un gateway o una piattaforma specifica e circoscritti a quella piattaforma; dall'altro i framework generici, sufficientemente astratti da operare su qualsiasi target ma che impiegano la specifica OpenAPI, quando disponibile, come documentazione di riferimento anziché come sorgente autoritativa da cui derivare a runtime l'intera conoscenza del target. Nessun framework disponibile nella letteratura e nel panorama degli strumenti open source combina agnosticismo applicativo con testing guidato dal contratto: la portabilità e la profondità semantica sono trattate come requisiti in tensione, invece di essere riconciliate attraverso la specifica formale del target come oracolo comune.

Il terzo gap riguarda la riproducibilità. Il penetration testing tradizionale è per natura point-in-time: condotto da un operatore in un momento specifico, con un insieme specifico di tecniche e conoscenze, il suo output riflette tanto le caratteristiche del target quanto le scelte di chi ha condotto la sessione. Un assessment il cui esito dipende da fattori non dichiarati e non controllabili rimane escluso dall'integrazione come quality gate in una pipeline Continuous Integration / Continuous Deployment (CI/CD): un quality gate richiede che la stessa condizione, valutata in momenti diversi sullo stesso target invariato, produca lo stesso esito, e un tool il cui output varia tra esecuzioni per ragioni non riconducibili a variazioni del target rimane al di fuori di questa classe di strumenti. La formalizzazione della riproducibilità come invariante empiricamente verificabile su esecuzioni successive, anziché come proprietà enunciata ma non dimostrata, costituisce uno dei problemi aperti nel security testing automatizzato che questo lavoro affronta direttamente.

1.3 Obiettivi della Ricerca

La risposta ai tre gap identificati è strutturata in tre obiettivi di ricerca corrispondenti, ciascuno formulato in modo da vincolare la soluzione a un meccanismo verificabile anziché a un'aspirazione di principio.

Il primo obiettivo affronta la cecità semantica degli scanner tradizionali: progettare e implementare un framework che utilizzi la specifica OpenAPI come oracolo formale, derivando a runtime tutta la conoscenza del target dalla specifica stessa e dal file di configurazione, senza riferimenti hardcoded a nessun sistema specifico. Il criterio di successo è verificabile per ispezione: l'assenza di qualsiasi literal dipendente dal target nel codice sorgente è la condizione che distingue un artefatto riutilizzabile da uno script di progetto. Il principio architetturale che governa questa scelta, l'agnosticismo applicativo, è discusso nella Sezione 4.1.1.

Il secondo obiettivo risponde all'assenza di strumenti che combinino portabilità e profondità semantica: dimostrare che l'approccio contract-driven supera strutturalmente i limiti del fuzzing cieco nell'identificazione di vulnerabilità logiche nelle API REST. La superiorità è strutturale perché deriva direttamente dall'uso della specifica come oracolo: un framework che costruisce richieste nel rispetto dei vincoli contrattuali raggiunge la logica semantica che rimane inaccessibile allo scanner sintattico, indipendentemente dal target specifico. Il meccanismo che realizza questa capacità è descritto nella Sezione 4.1.2.

Al terzo gap, la non confrontabilità del penetration testing tradizionale, corrisponde l'obiettivo di garantire la riproducibilità deterministica dell'assessment e di validarla empiricamente, producendo un tool integrabile in pipeline CI/CD tramite exit code semantici. La validazione empirica è parte costitutiva dell'obiettivo: enunciare la riproducibilità come proprietà attesa senza dimostrarla su esecuzioni successive riduce il claim a un'affermazione di principio. La riproducibilità deterministica è trattata nella Sezione 4.1.4; l'integrazione CI/CD tramite exit code semantici è discussa nella Sezione 7.2.1.

1.4 Organizzazione della Tesi

Il Capitolo 3 stabilisce il background teorico e la tassonomia delle minacce, analizzando i paradigmi delle architetture API moderne, il pattern del gateway come punto di enforcement della sicurezza, e l'evoluzione degli approcci di testing che hanno preceduto quello contract-driven. Il Capitolo 4 presenta la metodologia e le proprietà architetture fondamentali: stabilisce il perché delle scelte di design prima di descriverne i componenti, operando a livello di sistema. Il Capitolo 5 porta a livello implementativo quanto stabilito nel Capitolo 4, nominando classi, meccanismi Python specifici e la struttura della pipeline di esecuzione a sette fasi. Il Capitolo 6 valida empiricamente il sistema contro un target reale con dati tracciabili e verificabili. Il Capitolo 7 analizza i limiti strutturali dell'approccio e le traiettorie di sviluppo futuro. Il Capitolo 8 traccia la chiusura dell'arco argomentativo e colloca il contributo nel contesto della sicurezza come disciplina ingegneristica.

Capitolo 2

Background e Stato dell'Arte

Capitolo 3

Background e Stato dell'Arte

Il problema affrontato in questa tesi non è sorto nel vuoto: è il prodotto di una traiettoria evolutiva precisa, in cui la distribuzione della logica applicativa su reti di servizi autonomi ha moltiplicato la superficie esposta e reso obsoleti gli strumenti pensati per architetture monolitiche. Questo capitolo ricostruisce quella traiettoria in quattro movimenti. La Sezione 3.1 analizza i paradigmi architetturali e i protocolli di comunicazione che definiscono il perimetro delle API moderne. La Sezione 3.2 esamina il pattern dell'API gateway come punto di enforcement centralizzato della sicurezza nel paradigma Zero-Trust. La Sezione 3.3 classifica le categorie di minaccia rilevanti per le API REST esposte tramite gateway. La Sezione 3.4 analizza l'evoluzione degli approcci di testing, identificando i gap strutturali che motivano l'approccio contract-driven adottato nei capitoli successivi.

3.1 Paradigmi delle Architetture API Moderne

3.1.1 Evoluzione dei Sistemi Distribuiti

L'architettura monolitica, in cui tutta la logica applicativa risiede in un singolo processo deployabile, ha dominato lo sviluppo enterprise per decenni. Le sue proprietà sono note: deployment atomico, transazioni interne semplificate, toolchain di sviluppo consolidato. Il suo limite strutturale lo è altrettanto: la scala è ottenuta replicando l'intero monolite, indipendentemente da quale sottoinsieme funzionale sia effettivamente sotto carico, e ogni modifica richiede il redeployment dell'intera applicazione, aumentando il rischio di regressione con la crescita della codebase.

L'architettura a microservizi risponde a questo limite decomponendo il sistema in servizi autonomi, ciascuno responsabile di un sottodominio delimitato del business, deployabile e scalabile indipendentemente dagli altri [?]. La decomposizione ridistribuisce la

complessità: ogni servizio è più semplice del monolite, ma il sistema nel suo complesso introduce un nuovo ordine di problemi che il monolite non presentava. La comunicazione tra servizi avviene attraverso interfacce programmabili esposte in rete, ciascuna con il proprio contratto, il proprio ciclo di vita e la propria superficie di attacco. Il numero di queste interfacce cresce con le interazioni tra servizi, non linearmente con il numero di servizi in sé: un sistema a n servizi può esporre fino a $O(n^2)$ canali di comunicazione, ciascuno potenzialmente raggiungibile, interrogabile e abusabile indipendentemente dal contesto applicativo.

3.1.2 Tassonomia degli Stili Architetture e di Comunicazione

I servizi autonomi comunicano attraverso stili architetture con proprietà tecniche distinte, ciascuno con implicazioni di sicurezza proprie. La rassegna che segue è orientata a quelle proprietà, non alla sintassi dei protocolli.

Representational State Transfer (REST), formalizzato da Fielding nella sua dissertazione del 2000 [?], è uno stile architetturale stateless basato su risorse identificate da Uniform Resource Identifier (URI) e manipolate tramite verbi HTTP standard. La sua proprietà fondamentale è la separazione tra l'identificazione della risorsa e la sua rappresentazione: il client specifica quale risorsa vuole e quale operazione eseguire, il server decide come restituirla. Questa separazione genera una conseguenza di sicurezza strutturale: l'identificatore della risorsa è visibile nel path dell'Uniform Resource Locator (URL) e controllabile dal client, rendendo la verifica dell'autorizzazione a livello di oggetto una responsabilità applicativa che il protocollo non può enforce autonomamente.

Google Remote Procedure Calls (gRPC) adotta un modello diverso: le operazioni sono procedure nominate, i parametri sono tipizzati tramite Protocol Buffers, la serializzazione è binaria e il trasporto è HTTP/2. La tipizzazione forte e la serializzazione binaria chiudono la superficie a certi vettori di injection, ma introducono opacità al livello di ispezione del payload: un Web Application Firewall (WAF) che opera sulla semantica delle richieste HTTP/1.1 è strutturalmente cieco ai frame Protobuf che attraversano un tunnel HTTP/2.

Graph Query Language (GraphQL) espone un singolo endpoint che accetta query arbitrarie sul grafo dello schema: il client specifica esattamente i campi che vuole, compresi i percorsi di attraversamento del grafo. Questa flessibilità sposta il controllo dalla selezione degli endpoint alla validazione della complessità della query: una singola richiesta HTTP può contenere centinaia di query alias, consentendo di aggirare rate limiting configurato a livello di conteggio delle richieste HTTP.

Simple Object Access Protocol (SOAP), protocollo basato su Extensible Markup Language (XML) e contratti Web Services Description Language (WSDL), è prevalente in si-

stemi enterprise legacy. Le sue vulnerabilità strutturali derivano dal parsing XML: XML External Entity (XXE) injection e XML Bomb sono conseguenze dirette del modello di elaborazione del documento, indipendenti dall'applicazione specifica.

WebSocket e Server-Sent Events (SSE) condividono la proprietà di stabilire connessioni persistenti che sopravvivono al singolo ciclo request-response. Il rate limiting applicato sull'handshake HTTP iniziale non si estende ai frame successivi, e i meccanismi di ispezione del gateway operano esclusivamente sul traffico di upgrade, lasciando il canale post-handshake fuori dal perimetro di enforcement.

Questa tesi si focalizza su API REST documentate tramite specifica OpenAPI, per le ragioni discusse nella Sezione 1.2: è lo stile architetturale più adottato nelle architetture a microservizi cloud-native e quello per cui esiste una specifica formale sfruttabile come oracolo di assessment.

3.1.3 Il Fenomeno dell'API Sprawl

La decomposizione in microservizi, combinata con cicli di sviluppo rapidi e team decentralizzati, produce un effetto collaterale documentato: la proliferazione incontrollata di endpoint esposti che sfugge a qualsiasi inventario centralizzato [?]. Il fenomeno si manifesta in tre categorie distinte. Le *Shadow API* sono endpoint funzionanti ma assenti da qualsiasi specifica pubblicata: creati durante lo sviluppo, mai rimossi, raggiungibili ma non documentati. Le *Deprecated API* sono endpoint formalmente obsoleti che continuano a rispondere perché la loro disattivazione non è mai avvenuta o perché dipendenze non mappate li mantengono attivi. Le *Zombie API* sono endpoint di versioni precedenti del servizio che persistono in deployment non aggiornati.

Tutte e tre le categorie condividono una proprietà critica per il security assessment: sono invisibili agli strumenti che costruiscono la propria conoscenza del target a partire dalla documentazione. Un fuzzer che enumera gli endpoint dalla specifica OpenAPI non può raggiungere un endpoint non documentato; uno scanner che confronta le risposte con i codici attesi dalla specifica non può rilevare una Shadow API perché non sa che dovrebbe cercarvela. La rilevazione delle Shadow API richiede un approccio inverso: confrontare la superficie effettivamente raggiungibile con quella dichiarata, identificando la discrepanza. Questo è precisamente uno degli obiettivi del Domain-0 (API Discovery) descritto nella Sezione 4.3.1.

3.2 Il Pattern dell'API Gateway in Ottica Zero-Trust

3.2.1 Analisi Comparativa dei Modelli di Esposizione in Rete

Le architetture a microservizi dispongono di quattro modelli principali per esporre i propri servizi al traffico esterno e per governare il traffico interno, ciascuno con un diverso posizionamento nel perimetro di sicurezza.

L'API gateway enterprise è il punto di terminazione centralizzato per il traffico nord-sud, ovvero il traffico proveniente da client esterni. Concentra in un unico strato le funzioni trasversali che altrimenti andrebbero replicate in ogni servizio: autenticazione e autorizzazione, rate limiting, trasformazione dei payload, terminazione Transport Layer Security (TLS) e logging delle transazioni. La centralizzazione rende il gateway il punto naturale in cui enforcare le policy di sicurezza prima che le richieste raggiungano la logica applicativa; è anche il punto in cui un errore di configurazione produce conseguenze sistemiche, non localizzate a un singolo servizio.

Il modello Kubernetes Ingress e Gateway API governa il traffico nord-sud all'interno di un cluster containerizzato. La risorsa Ingress definisce regole di routing dichiarative (hostname, path, servizio di destinazione) che un Ingress Controller traduce in configurazione del reverse proxy sottostante. La Kubernetes Gateway API, introdotta come standard stabile nel 2023, risolve le limitazioni di portabilità del modello Ingress sostituendo le annotation controller-specific con risorse tipizzate e role-oriented. Dal punto di vista del security assessment, il gateway gestisce il traffico di upgrade e l'autenticazione iniziale, ma la sua capacità di ispezione si limita al piano HTTP: traffico WebSocket post-handshake e payload Protobuf attraversano il layer senza ispezione del contenuto.

Il service mesh governa il traffico est-ovest, ovvero le comunicazioni inter-servizio all'interno del perimetro. Architetture basate su sidecar proxy (Envoy nel caso di Istio) intercettano tutto il traffico del pod in modo trasparente, abilitando mutua autenticazione TLS (mTLS), osservabilità e circuit breaking senza modifiche al codice applicativo. La mesh introduce però assunzioni implicite sulla natura del traffico: un sidecar che bufferizza messaggi per implementare retry e metriche assume che i flussi siano finiti e di durata limitata; uno stream gRPC bidirezionale indefinito viola questa assunzione e può produrre esaurimento di memoria nel proxy.

Il modello serverless delega la gestione del ciclo di vita dei container alla piattaforma cloud. Il traffico è gestito da un gateway managed che scala automaticamente e non richiede configurazione infrastrutturale esplicita. Il costo di questa astrazione è visibile sul piano della sicurezza: l'accesso amministrativo al piano di controllo è mediato da policy IAM della piattaforma e non è disponibile nella forma di un'Admin API ispezionabile, il che rende inapplicabili le verifiche `WHITE_BOX` descritte nella Sezione 4.3.2.

3.2.2 Criteri di Selezione e Giustificazione Architetturale

Dei quattro modelli analizzati, l'API gateway enterprise è l'unico che soddisfa congiuntamente i requisiti del framework proposto in questa tesi. Tre criteri guidano questa selezione.

Il primo criterio è la verificabilità del piano di controllo. Un gateway enterprise espone un'Admin API che consente l'ispezione diretta della configurazione: plugin attivi, policy di routing, parametri di rate limiting, variabili d'ambiente. Questa interfaccia è il presupposto tecnico dei test `WHITE_BOX`, che verificano garanzie non osservabili dall'esterno tramite probe empirici. Il service mesh delega parte di questa visibilità al control plane distribuito; il modello serverless la sottrae interamente.

Il secondo criterio è la separazione tra piano di autenticazione e logica applicativa. Un gateway che centralizza l'enforcement delle policy di autenticazione e autorizzazione consente di valutare questi meccanismi indipendentemente dall'applicazione ospitata: un test che verifica il rifiuto delle richieste non autenticate non deve conoscere la logica applicativa, deve conoscere solo il contratto che il gateway dovrebbe enforce. Questa separazione è la condizione che rende il framework applicazione-agnostico nel senso stabilito nella Sezione 4.1.1.

Il terzo criterio è la documentazione formale delle API esposte. Un gateway enterprise instrada il traffico verso applicazioni che espongono la propria interfaccia tramite specifica OpenAPI; questa specifica è la sorgente da cui il framework deriva a runtime tutta la conoscenza del target. I modelli serverless e mesh non escludono questa possibilità, ma non la presuppongono come requisito architetturale.

3.2.3 Enforcing del Paradigma Zero-Trust

Il paradigma Zero-Trust, formalizzato nel NIST Special Publication 800-207 [?], ridefinisce il perimetro di sicurezza spostando il punto di controllo dalla rete all'identità: nessuna richiesta è considerata attendibile per via della sua provenienza da una rete ritenuta fidata, e ogni accesso è valutato in modo indipendente rispetto all'identità del chiamante, al contesto della richiesta e ai permessi sulla risorsa specifica. Applicato alle API REST, il principio si traduce in tre requisiti operativi: autenticazione obbligatoria su ogni endpoint, autorizzazione valutata per ogni coppia chiamante-risorsa, e assenza di fiducia implicita derivante dalla posizione di rete del client.

L'API gateway è il punto architetturale in cui questi requisiti trovano enforcement naturale: si colloca tra il client e il backend, ha visibilità su ogni richiesta in transito, e può rifiutare le richieste che non soddisfano i controlli configurati prima che raggiungano la logica applicativa. La verifica che questo enforcement sia effettivamente attivo e cor-

rettamente configurato è esattamente il perimetro del framework proposto: le garanzie che il gateway dichiara nella propria configurazione (autenticazione richiesta su tutti gli endpoint, rate limiting attivo, header di sicurezza iniettati) devono corrispondere al comportamento osservabile a runtime, e questa corrispondenza non è verificabile senza un assessment sistematico. La distanza tra configurazione dichiarata e comportamento effettivo è il gap che motiva il dominio Configuration & Hardening (Domain-6) e i test `WHITE_BOX` della Sezione 4.3.2.

3.3 Tassonomia delle Minacce e Vulnerabilità Logiche

3.3.1 Il Semantic Gap e l'Evoluzione del Panorama delle Minacce

La transizione verso architetture a microservizi ha spostato il baricentro delle vulnerabilità dalle anomalie sintattiche alle violazioni semantiche. Un WAF tradizionale intercetta una SQL injection perché riconosce nel payload un pattern sintattico noto; la stessa infrastruttura è strutturalmente cieca a una richiesta `GET /users/456` perfettamente formata inviata da un utente che ha il permesso di leggere solo `/users/123`, perché quella violazione non si esprime nella forma della richiesta ma nella relazione tra l'identità del chiamante e la risorsa richiesta. Il WAF ispeziona il protocollo; la vulnerabilità risiede nel contratto applicativo.

Questa distanza tra ciò che uno strumento di testing può osservare e ciò che una vulnerabilità semantica richiede di comprendere costituisce il *semantic gap*: il divario strutturale tra la forma della transazione HTTP e la semantica del contratto che quella transazione dovrebbe rispettare. Il framework OWASP API Security Top 10 nella sua edizione 2023 [?] formalizza questa transizione: le prime cinque categorie di rischio, Broken Object Level Authorization (API1), Broken Authentication (API2), Broken Object Property Level Authorization (API3), Unrestricted Resource Consumption (API4) e Broken Function Level Authorization (API5), sono tutte vulnerabilità semantiche. La loro manifestazione comune è una richiesta sintatticamente corretta che viola le assunzioni implicite del sistema sull'identità, i privilegi o il comportamento atteso del chiamante.

Colmare il semantic gap richiede che lo strumento di assessment possieda una rappresentazione formale del contratto API e la utilizzi come oracolo per valutare le risposte ricevute. La specifica OpenAPI è quella rappresentazione: definisce quali endpoint esistono, quali parametri accettano, quali schemi di risposta producono e quali codici di stato sono attesi. Uno strumento che costruisce le proprie richieste nel rispetto di questa specifica e valuta le risposte rispetto a ciò che il contratto prevede per quella specifica operazione è in grado di raggiungere la classe di vulnerabilità che i fuzzer sintattici lasciano intatta. Questa è la premessa teorica dell'approccio adottato, il cui meccanismo implementativo è descritto nella Sezione 4.1.2.

3.3.2 Categorie di Fallimento Architetturale

Le vulnerabilità semantiche delle API REST si raggruppano in tre macro-categorie distinte per natura del fallimento, in ordine crescente di complessità del vettore di attacco.

La prima categoria racchiude i difetti di autorizzazione e gestione dell'accesso agli oggetti. Il Broken Object Level Authorization (BOLA, API1:2023) si manifesta quando un sistema verifica che il chiamante sia autenticato ma non verifica che abbia il diritto di accedere alla specifica risorsa identificata dal path. La richiesta è formalmente corretta; il fallimento risiede nell'assenza o nell'inadeguatezza del controllo sulla coppia chiamante-oggetto. Il Broken Function Level Authorization (Broken Function Level Authorization (BFLA), API5:2023) segue la stessa logica a livello di operazione: un utente con un ruolo ristretto riesce a invocare un endpoint riservato a ruoli privilegiati, tipicamente perché i controlli di autorizzazione sono applicati in modo non uniforme tra endpoint dello stesso servizio. Entrambe le classi richiedono, per essere rilevate in modo sistematico, la capacità di effettuare richieste con credenziali di ruoli distinti e confrontarne le risposte: un workflow multi-step che i fuzzer stateless non supportano.

La seconda categoria comprende le violazioni del principio di minimo privilegio nel flusso dei dati. Il mass assignment si verifica quando un endpoint accetta nel payload di richiesta campi non documentati nello schema contrattuale, che il backend mappa direttamente sul modello di dominio senza whitelist esplicita: un attaccante può elevare i propri privilegi o modificare attributi riservati semplicemente includendo campi aggiuntivi in un POST o PUT. L'esposizione eccessiva di dati (API3:2023) è la controparte nella risposta: il servizio restituisce campi sensibili, tra cui hash di password, chiavi di API o dati personali identificativi, perché il livello applicativo serializza l'intero oggetto di dominio affidando al client la selezione dei campi rilevanti. Entrambi i fallimenti sono invisibili a uno strumento che non conosce lo schema contrattuale atteso: a livello HTTP, la transazione appare corretta in entrambi i sensi.

La terza categoria abbraccia l'abuso della topologia distribuita come vettore di attacco indiretto. Il Server-Side Request Forgery (SSRF) (API7:2023) sfrutta endpoint che accettano URL forniti dal client, come webhook, mirror o proxy, per far sì che il server effettui richieste verso infrastrutture interne o verso i metadata endpoint dei cloud provider, esfiltrando credenziali di esecuzione o raggiungendo servizi non esposti al perimetro esterno. L'assenza di rate limiting (API4:2023) espone il sistema all'esaurimento controllato delle risorse: un attaccante che può inviare richieste senza limitazione può saturare il backend, i pool di connessioni al database o la capacità computazionale, rendendo il servizio indisponibile per gli utenti legittimi. Entrambe le classi operano attraverso funzionalità legittime del sistema, non attraverso payload malformati, e sono quindi sistematicamente invisibili ai layer di ispezione sintattica.

3.3.3 Dal Modello Teorico alla Necessità di Valutazione Strutturata

Le tre categorie identificate nella sezione precedente non sono omogenee né per il tipo di preconditione che richiedono né per il meccanismo di rilevazione che presuppongono. Rilevare una violazione BOLA richiede token validi per almeno due identità distinte e la capacità di confrontare le risposte ricevute con credenziali diverse. Rilevare mass assignment richiede di conoscere lo schema di richiesta contrattuale e di verificare il comportamento del sistema in presenza di campi non dichiarati. Rilevare un SSRF richiede endpoint che accettino URL e un meccanismo per osservare le richieste che il server effettua verso l'esterno. Nessuno dei tre fallimenti è raggiungibile con lo stesso strumento e la stessa configurazione.

Questa eterogeneità è la ragione per cui un singolo scanner generico non è sufficiente: le classi di vulnerabilità semantica impongono preconditioni di accesso distinte, richiedono strategie di osservazione diverse e non si mappano sulla stessa superficie di attacco. La risposta architetturale a questa eterogeneità è la tassonomia a otto domini di assurance descritta nella Sezione 4.3.1, in cui ogni dominio raggruppa le garanzie che condividono le stesse preconditioni informative e lo stesso tipo di evidenza necessario per la verifica. Il box gradient, discusso nella Sezione 4.3.2, formalizza come il livello di visibilità disponibile determini la classe di garanzie raggiungibile: la struttura di preconditioni che emerge dall'analisi delle minacce in questa sezione non è un ostacolo da aggirare, ma il criterio che orienta la classificazione dei test nel Capitolo 4.

3.4 L'Evoluzione del Testing di Sicurezza: dall'Analisi Cieca ai Contratti

3.4.1 Paradigmi Tradizionali e i Limiti del DAST/SAST

Gli approcci classici al security testing delle applicazioni web si dividono in due famiglie distinte per punto di osservazione. Il Static Application Security Testing (SAST) analizza il codice sorgente o i binari compilati senza eseguire l'applicazione, identificando pattern noti di vulnerabilità a livello sintattico: buffer overflow, uso di funzioni non sicure, flussi di dati non validati verso sink pericolosi [?]. Il DAST interagisce con l'applicazione in esecuzione attraverso la sua interfaccia esterna, inviando richieste e osservando le risposte alla ricerca di comportamenti anomali: codici di errore rivelatori, stack trace nelle risposte, variazioni nei tempi di risposta correlate a payload specifici.

Entrambi i paradigmi, applicati alle API REST, incontrano un limite strutturale prima ancora di raggiungere la logica applicativa. Il DAST genera payload a partire da euristiche sintattiche: variazioni di parametri, payload di injection, valori ai limiti degli

intervalli attesi. Questi payload vengono costruiti senza conoscere il contratto che l'endpoint dichiara; di conseguenza, una quota sostanziale delle richieste viola i tipi, i formati o i vincoli enumerati nello schema e viene respinta con HTTP 400 dal layer di validazione dell'input, prima che qualsiasi logica di business venga valutata. Questo fenomeno, denominato *combinatorial wall*, è la barriera strutturale che impedisce al fuzzing cieco di raggiungere sistematicamente la classe di vulnerabilità semantica descritta nella Sezione 3.3.2: lo spazio di tutti i payload sintatticamente possibili è astronomico, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile, e soltanto quest'ultimo è in grado di interrogare la logica applicativa. Il SAST, dal canto suo, analizza il codice ma non può osservare il comportamento a runtime in presenza di identità autenticate con ruoli distinti: una violazione BOLA emerge dall'interazione tra credenziali, routing del gateway e logica di autorizzazione, non da un pattern sintattico nel codice sorgente isolato.

Lo strumento DAST più diffuso nel contesto delle API, OWASP ZAP [?], è progettato per operare su applicazioni web generiche e supporta API REST tramite importazione della specifica OpenAPI. Anche in questa modalità, tuttavia, ZAP utilizza la specifica per enumerare gli endpoint e costruire richieste valide, ma valuta le risposte rispetto a euristiche generiche di anomalia, non rispetto a ciò che il contratto prevede per quella specifica operazione su quella specifica risorsa con quelle specifiche credenziali. La distinzione è sostanziale: un campo sensibile restituito in una risposta 200 è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso; uno scanner privo di quella conoscenza non può rilevarlo come anomalia.

3.4.2 Asimmetria Informativa e il Box Gradient

La capacità di rilevare una vulnerabilità non dipende solo dalla sofisticazione dello strumento, ma dalla quantità di informazione che lo strumento possiede sul sistema sotto test prima di iniziare la valutazione. Questa asimmetria informativa è il principio che sottende la classificazione Black/Grey/White Box nel testing del software [?]: un tester che non ha accesso al codice sorgente né alla configurazione interna del sistema opera nel regime Black Box, limitando la propria osservazione al comportamento esterno; un tester con accesso completo alla configurazione interna opera nel regime White Box, potendo verificare per ispezione diretta proprietà non osservabili dall'esterno.

Applicata alle API REST esposte tramite gateway, questa distinzione acquista una precisione tecnica che va oltre la semplice metafora della visibilità. Alcune garanzie di sicurezza sono fisicamente raggiungibili solo in determinati regimi di visibilità: verificare che il gateway rifiuti le richieste non autenticate non richiede nulla più dell'accesso di rete, perché il gateway applica quel controllo a qualsiasi interlocutore; simulare un attaccante esterno privo di credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti. Verificare una violazione BOLA richiede almeno due

identità con ruoli distinti, entrambe autenticate: senza quello stato il gateway respinge la richiesta prima che la logica di autorizzazione venga valutata, e l'esito è indistinguibile da un corretto rifiuto di autenticazione. Verificare che le variabili d'ambiente del gateway non contengano credenziali in chiaro richiede accesso al piano di configurazione amministrativo: nessun probe empirico verso gli endpoint esposti può restituire quella informazione.

Ne deriva che il gradiente Black/Grey/White Box, nell'approccio adottato in questa tesi, non è una tassonomia teorica applicata retroattivamente ai test, ma il prodotto delle precondizioni concrete che ciascuna garanzia di sicurezza impone al tester. La sua formalizzazione come proprietà architetturale (descritta nella Sezione 4.3.2) è la conseguenza diretta di questa analisi: assegnare a un test un livello di visibilità senza derivarlo dalle sue precondizioni produce una classificazione decorativa, non operativa.

3.4.3 Contract-Driven Security e Approcci Ibridi

Il limite strutturale del fuzzing cieco, l'incapacità di costruire sistematicamente richieste valide rispetto al contratto senza possedere il contratto stesso, ha motivato una linea di ricerca che utilizza le specifiche formali delle API come punto di partenza per la generazione dei test. RESTler [?], il primo fuzzer stateful per API REST presentato all'ICSE 2019, analizza la specifica OpenAPI del servizio target e inferisce le dipendenze produttore-consumatore tra i tipi di richiesta: se un endpoint `POST /resources` produce un `resource_id` che un endpoint `GET /resources/{id}` consuma come parametro di path, RESTler genera automaticamente la sequenza nell'ordine corretto. Questo approccio abbatte il combinatorial wall costruendo richieste strutturalmente valide a partire dal contratto e consentendo al fuzzer di raggiungere la logica applicativa che il fuzzing cieco non raggiunge.

RESTler dimostra la praticabilità tecnica del paradigma contract-driven, ma il suo obiettivo primario è la generazione di sequenze di test per trovare crash e comportamenti inattesi, non la verifica sistematica di garanzie di sicurezza specifiche. La specifica OpenAPI viene utilizzata per costruire le richieste, ma le risposte sono valutate rispetto a euristiche di anomalia generali, non rispetto a ciò che il contratto prevede per quella combinazione di operazione, risorsa e identità del chiamante. Ne deriva che RESTler è efficace nell'identificare difetti di robustezza e nell'esplorare lo spazio degli stati raggiungibili, ma non è progettato per verificare se una risposta 200 contenga campi che il contratto non prevede, se un token revocato venga ancora accettato, o se un endpoint di configurazione del gateway esponga credenziali non cifrate.

Il gap che rimane aperto è precisamente quello che questa tesi intende colmare: un framework che utilizzi la specifica OpenAPI non solo per costruire richieste valide ma come oracolo formale per valutare la correttezza semantica delle risposte, che operi in modo

Capitolo 3 Background e Stato dell'Arte

agnostico rispetto al target applicativo specifico, e che garantisca la riproducibilità deterministica dei risultati come requisito verificabile empiricamente. Il Capitolo 4 presenta l'architettura che realizza questi tre obiettivi, a partire dai principi fondamentali che li rendono possibili congiuntamente.

Capitolo 4

Metodologia, Architettura e Principi di Design

APIGuard Assurance è un framework di security assessment automatizzato per API REST esposte tramite gateway, progettato per essere applicazione-agnostico, contract-driven e deterministicamente riproducibile. Nasce dalla necessità di superare la frammentazione degli approcci manuali e target-specifici: strumenti che richiedono di essere riscritti per ogni nuovo target, che producono risultati non confrontabili tra esecuzioni, e che non possono essere integrati in processi di verifica continuativi. Il suo obiettivo è produrre un assessment sistematico, tracciabile e ripetibile, applicabile a qualsiasi API REST documentata senza modifiche al codice.

La presentazione di *APIGuard Assurance* segue un approccio *top-down*: dai principi generali di sistema che ne definiscono il perimetro fino alle scelte implementative che li concretizzano. Architettura e implementazione sono trattate in capitoli distinti perché operano su livelli di astrazione distinti. Il Capitolo 5 mostra come i principi descritti vengono implementati.

Il design di *APIGuard Assurance* è formalizzato in 39 proprietà architetture distribuite su sette categorie; la Tabella 4.1 le raccoglie come mappa di riferimento.¹

4.1 Principi Fondamentali: il Perché delle Scelte Architetture

In coerenza con la progressione *top-down* dichiarata, l'analisi ha inizio dal livello di massima astrazione: le decisioni che definiscono il carattere del tool prima ancora di

¹In grassetto le proprietà trattate in questo capitolo.

Dominio	Proprietà
D1: Architettura & Design	D1.P1, D1.P2, D1.P3 , D1.P4, D1.P5, D1.P6
D2: Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3: Config & Riproducibilità	D3.P1, D3.P2, D3.P3, D3.P4, D3.P5
D4: Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5: Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6: CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7: Packaging & Deployment	D7.P1, D7.P2, D7.P3

Tabella 4.1: Proprietà APIGuard Assurance per dominio.

descrivere i componenti. Le quattro proprietà discusse in questa sezione costituiscono la struttura portante del sistema: le scelte architetturali trattate nelle sezioni successive si stratificano su di esse, ciascuna risolvendo una frizione specifica all'interno dei vincoli che queste proprietà fondamentali stabiliscono.

4.1.1 Agnosticismo Applicativo (D1.P1)

Quali target applicativi è possibile valutare senza modificare il codice del tool?

Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi del deployment (descritti in dettaglio nella Sezione 4.1.3), e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, la definizione dei parametri di path, query e header per ciascuna operazione, gli schemi dei payload di richiesta e risposta, e i codici di stato attesi. Ne consegue un vincolo strutturale preciso: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica, e qualsiasi API REST documentata è testabile senza modifiche al codice.

Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione produce **SKIP**, lo stato previsto per uno scope legittimamente vuoto, distinto dall'**ERROR** che segnalerebbe un malfunzionamento interno al tool. Il principio si estende a qualsiasi requisito non soddisfatto: un tool esterno non installato, la Admin API del gateway non raggiungibile, un insieme di endpoint privi di seed configurato; ciascuna di queste condizioni produce **SKIP** sui test che ne dipendono e lascia proseguire gli altri, a meno che il risultato non costituisca una violazione confermata di un controllo perimetrale fondamentale (come l'assenza totale di autenticazione su tutti gli endpoint esposti), caso in cui la pipeline può essere interrotta esplicitamente tramite configurazione.

La differenza tra uno script target-specifico e un tool di assessment generale non è di complessità, ma di generalità: uno script con path e strutture dati scritti nel codice funziona su quel target e solo su quello (sostituita l'applicazione, va riscritto), mentre un tool che deriva tutta la propria conoscenza dalla specifica formale del target è applicabile a qualsiasi API REST documentata senza modifiche. Questa generalità è precisamente il confine che separa un artefatto di progetto da un contributo riutilizzabile.

Il costo accettato è la dipendenza da una specifica valida. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: il tool non ha modo di distinguere un'assenza di vulnerabilità da un'assenza di copertura, perché l'unico oracolo di cui dispone è la specifica stessa. Questa tensione costituisce il limite strutturale discusso nella Sezione 7.1 e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale.

4.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)

Quale strumento formale governa la costruzione delle richieste e la valutazione delle risposte nell'intera pipeline?

La specifica OpenAPI è l'unico vincolo contrattuale che governa l'intera pipeline: costruisce ogni richiesta nel rispetto dei tipi, dei formati e dei parametri dichiarati, delimita la superficie di attacco agli endpoint documentati e valuta ogni risposta rispetto a ciò che il contratto prevede per quella specifica operazione.

Un fuzzer che genera payload senza conoscere il contratto affronta un ostacolo strutturale prima ancora di raggiungere la logica applicativa: una quota sostanziale delle richieste viene respinta con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nel Capitolo 3: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile. Usare la specifica come oracolo formale abbatte quella barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo

sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé una dipendenza dalla qualità della specifica, già enunciata per D1.P1 (Agnosticismo Applicativo), ma con una conseguenza distinta. In D1.P1 il problema era di copertura: una specifica incompleta riduce gli endpoint osservabili senza che il tool possa rilevarlo. Qui il problema è di affidabilità dei verdeti: una specifica contrattualmente errata produce valutazioni delle risposte basate su un oracolo sbagliato, e il tool non ha modo di distinguere un contratto corretto da uno incorretto. Si tratta del paradosso del Ground Truth, discusso nella Sezione 7.1 come limite strutturale del paradigma contract-driven e non come anomalia specifica di questa implementazione.

4.1.3 Config-Driven Development (D3.P1)

Esiste un'unica fonte di verità per tutti i parametri che governano il comportamento del tool?

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. Tutti i parametri operativi risiedono nel file `config.yaml`, strutturato in aree funzionali con responsabilità distinte: la localizzazione del gateway e della specifica OpenAPI; i parametri di esecuzione globali, tra cui soglia di priorità minima, strategia di filtraggio dei test, timeout per singola esecuzione e formato del logging; il dizionario `path_seed`, che associa a ogni parametro di path parametrico un valore concreto presente sul target; le configurazioni specifiche per singolo test, organizzate per dominio; e i parametri degli strumenti di analisi esterni, con abilitazione per tool, versione attesa e opzioni di invocazione. Le credenziali di autenticazione, per loro natura non versionabili, risiedono in un file `.env` separato, il cui contenuto viene interpolato dal loader prima del parsing. Il codice sorgente non contiene nessun literal decisionale hardcoded: qualsiasi parametro che possa variare tra un'esecuzione e l'altra è dichiarato esplicitamente e validato prima dell'avvio della pipeline.

La separazione tra logica e configurazione assegna a ogni variazione di comportamento un'unica origine dichiarata: il file `config.yaml` attivo durante quella run. Se il comportamento del tool cambia tra un'esecuzione e l'altra, è perché è cambiata la configurazione; se la configurazione è identica, l'output è identico. Questa centralizzazione è anche la condizione che rende possibile la proprietà discussa nella sottosezione seguente: un sistema che legga valori da variabili d'ambiente non dichiarate, da file di stato impliciti o da sorgenti runtime non controllate non può garantire che due esecuzioni successive producano lo stesso risultato, indipendentemente dalla correttezza della logica di test. Fissare la configurazione è il primo passo necessario per fissare l'output.

4.1.4 Riproducibilità Deterministica (D3.P4)

Come viene garantito che l'assessment produca risultati confrontabili tra esecuzioni successive?

Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati identici a ogni esecuzione. La riproducibilità è il presupposto che conferisce valore dimostrativo ai risultati del Capitolo 6: senza di essa, ogni run è un'osservazione singola non confrontabile, e l'assessment non è verificabile da terzi.

Il determinismo è ottenuto attraverso tre meccanismi distinti che si completano a vicenda:

- D3.P1 (Config-Driven Development): se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene.
- Struttura del DAG, trattata nella Sezione 4.2.2: un grafo aciclico orientato con risoluzione topologica deterministica produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.
- Ordinamento lessicografico dei test all'interno di ogni batch del DAG: l'ordinamento topologico puro non garantisce stabilità tra test pronti allo stesso livello, e senza un criterio di tie-breaking esplicito la sequenza potrebbe variare tra esecuzioni; l'ordine lessicografico risolve questa ambiguità, garantendo che la sequenza sia sempre identica a parità di input.

La garanzia di riproducibilità non riguarda ogni componente dell'output: timestamp e identificatori univoci dei singoli record di evidenza variano per natura tra esecuzioni. Ciò che deve essere invariante sono i contatori aggregati (PASS, FAIL, SKIP), i verdetti per singolo test e la distribuzione dei finding per dominio. La validazione empirica, condotta su due run indipendenti che hanno prodotto contatori aggregati byte-equivalenti (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, con `diff status per-test = 0` e delta wall-clock inferiore allo 0,3%), è discussa nella Sezione 6.3.

4.2 Architettura dell'Assessment Engine

Le proprietà discusse nella sezione precedente costituiscono la base del sistema. Le scelte di questa sezione si aggiungono ad esse, ciascuna rispondendo a un'esigenza architetturale specifica che le proprietà fondamentali lasciano aperta.

4.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Il grafo delle dipendenze tra strati è strettamente aciclico e orientato in una sola direzione: il nucleo del sistema è importato dagli strati di integrazione, che sono importati dagli strati di test, che sono importati dall'orchestratore. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere il nucleo; aggiungere un connector non può influenzare l'orchestratore; l'intera base del nucleo può essere testata unitariamente senza istanziare alcun client HTTP né alcun test di sicurezza.

In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito nell'orchestratore, e così via. La direzione è unidirezionale: un test può dipendere dal nucleo per accedere ai componenti condivisi, ma nessun modulo del nucleo può dipendere da un test. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo è verificabile per ispezione del grafo di importazione: nessun modulo dello strato di test importa componenti degli strati superiori, e ogni strato dichiara esplicitamente nel proprio contratto le sole dipendenze che gli sono ammesse. I dettagli di come questo vincolo è garantito a livello di codice sono descritti nella Sezione ??.

La struttura a strati rende praticabile uno degli sviluppi futuri descritti nella Sezione 7.3: un modulo di test contribuito da terze parti viene scoperto automaticamente² senza che nessuna modifica al nucleo del sistema sia necessaria, perché il nucleo non contiene nessun riferimento ai test che lo utilizzano.

4.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine di esecuzione dei test è calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente come parte del proprio contratto. Lo scheduler raccoglie queste dichiarazioni, costruisce un DAG orientato e risolve l'ordine topologico, producendo una sequenza di batch: gruppi di test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch i test sono privi di dipendenze reciproche. La traduzione di questo schema in codice, incluse le strutture dati e le librerie impiegate, è descritta nella Sezione 5.2.2.

²La scoperta automatica non è priva di vincoli: un modulo viene incluso nella pipeline solo se rispetta i contratti del tool, ovvero se dichiara tutti gli attributi obbligatori e si colloca nella directory attesa. Un modulo che non soddisfi questi requisiti viene ignorato silenziosamente oppure produce un errore di validazione.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JSON Web Token (JWT) presuppone che il meccanismo di autenticazione funzioni: se eseguito prima del test di autenticazione, il token non è disponibile nel contesto condiviso e il test fallirebbe per la ragione sbagliata. Senza una dichiarazione esplicita della dipendenza, la disponibilità del token è una preconditione implicita che dipende dall'ordine di esecuzione scelto: un cambio di scheduling la viola silenziosamente. Con la dipendenza dichiarata, il DAG verifica la preconditione a monte dell'esecuzione, e un token non disponibile diventa un errore rilevabile prima che un singolo test venga avviato.

Lo scheduler include due proprietà di correttezza strutturale:

- Rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, l'errore viene segnalato prima che un singolo test venga eseguito, impedendo che stati inconsistenti si propaghino silenziosamente nella pipeline.
- Salvaguardia contro lo stallo: se il grafo risulta ancora attivo ma non riesce a estrarre nessun test pronto, il sistema emette un errore strutturato che elenca i test mai schedulati e interrompe l'esecuzione in modo controllato, fornendo all'operatore la diagnostica necessaria senza dover ricostruire lo stato interno del grafo.

L'ordinamento topologico puro non garantisce un ordine unico tra test pronti allo stesso livello: senza un criterio di tie-breaking³ esplicito la sequenza potrebbe variare tra esecuzioni. Lo scheduler risolve questa ambiguità ordinando lessicograficamente i test all'interno di ogni batch, convertendo la struttura parzialmente ordinata del grafo in una sequenza riproducibile, condizione necessaria per D3.P4 (Reproducibility).

La struttura a batch porta con sé una proprietà che la versione attuale non sfrutta: i test all'interno di uno stesso batch non hanno dipendenze reciproche e sono concettualmente parallelizzabili. Introdurre esecuzione parallela all'interno di un batch è una modifica localizzata allo scheduler, senza impatto sul design dei singoli test, a condizione che le implicazioni di concorrenza sullo stato condiviso vengano analizzate formalmente. Questa traiettoria è documentata come sviluppo futuro nella Sezione 7.3; la sequenzialità attuale è una scelta deliberata di scope, non un vincolo architetturale.

4.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile introduce

³ *Tie-breaking*: regola di ordinamento applicata tra elementi a parità di livello topologico, per garantire una sequenza stabile e riproducibile tra esecuzioni.

una categoria di bug difficile da diagnosticare: un test che scrive accidentalmente sulla rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte:

- **TargetContext** - immutabile per costruzione: viene popolato una volta sola prima che la pipeline inizi e qualsiasi tentativo di modifica produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che si conosce del target prima dell'esecuzione:
 - la specifica OpenAPI dereferenziata come mappa strutturata di endpoint, parametri e schemi;
 - i parametri di tuning specifici per ogni test ricavati dalla configurazione;
 - l'adapter del gateway iniettato prima dell'avvio della pipeline;
 - le credenziali, risolte dal loader a partire dal file `.env` e congelate nel **TargetContext** tramite la configurazione `frozen`.⁴
- **TestContext** - mutabile, raccoglie ciò che la pipeline produce. L'accesso è strutturato in tre canali tipizzati con responsabilità distinte, anziché in un dizionario ad accesso libero:
 - canale dei token: organizza le credenziali per ruolo, con accesso tramite un identificatore definito dal contratto del sistema, non tramite chiavi stringa arbitrarie;
 - canale di teardown: mantiene un registro Last In, First Out (LIFO) delle risorse create durante la run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.5.3;
 - canale dei dati condivisi: consente a test in batch successivi di consumare risultati prodotti da batch precedenti, senza che tra i moduli vengano introdotte dipendenze di importazione.

L'implementazione concreta di questa separazione, inclusi i tipi e i meccanismi di accesso, è descritta nella Sezione 5.1.3.

⁴L'accesso avviene esclusivamente tramite i campi tipizzati del modello Pydantic costruito a runtime: non esiste un dizionario libero né accesso diretto ai valori grezzi.

Qualsiasi informazione scoperta durante l'esecuzione (endpoint presenti sul target ma non dichiarati nella specifica, token acquisiti, risorse create) appartiene per definizione allo stato dell'assessment e non alla conoscenza preliminare del target: viene quindi registrata nel contesto di esecuzione, non in quello del target immutabile. La separazione fa sì che la base informativa da cui ogni verdetto dipende non possa essere alterata durante la run, condizione necessaria perché D3.P4 (Reproducibility) sia dimostrabile empiricamente e non solo enunciabile come proprietà attesa.

4.2.4 Gateway-Agnostic Adapter (D1.P6)

D1.P1 stabilisce che la conoscenza del target applicativo deriva esclusivamente dalla specifica OpenAPI e dal `config.yaml`. La proprietà presentata in questa sezione estende lo stesso principio di agnosticismo a un livello distinto, lo strato di controllo del gateway: alcune garanzie di sicurezza, come la verifica che i timeout sui servizi upstream siano configurati, che il plugin di circuit breaking sia attivo o che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente, non sono osservabili interrogando gli endpoint esposti ai client. Queste informazioni risiedono nel piano di configurazione amministrativo del gateway e richiedono un canale di accesso separato dai normali probe HTTP.

Il pattern scelto è un'interfaccia astratta di sola lettura verso il gateway: i test `WHITE_BOX` vi accedono esclusivamente tramite i metodi che questa interfaccia espone, senza contenere nessun riferimento al gateway specifico installato nel deployment. L'implementazione concreta viene selezionata a runtime in base alla configurazione e iniettata nel contesto del target prima che la pipeline abbia inizio. Se la Admin API del gateway non è configurata o non è raggiungibile, tutti i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita, senza errori a cascata. I dettagli dell'interfaccia e dell'implementazione Kong sono descritti nella Sezione 5.5.1.

Il vantaggio è che aggiungere supporto per un gateway diverso richiede di implementare una nuova classe concreta che soddisfi l'interfaccia, senza toccare nessuno dei test che la utilizzano: la logica di verifica non cambia, cambia solo il meccanismo con cui i dati di configurazione vengono recuperati. Il costo è che l'interfaccia può esprimere solo le verifiche comuni a tutti i gateway supportati: controlli che richiedano funzionalità vendor-specific non generalizzabili rimangono fuori dal perimetro dell'astrazione, e questo costituisce uno dei limiti strutturali discussi nella Sezione 7.1.

4.2.5 Streaming Evidence Store (D4.P1)

Un finding prodotto da un test acquista valore dimostrabile solo se accompagnato dall'evidenza HTTP che lo supporta: la request inviata, la response ricevuta, il timestamp e

gli identificatori di traccia. Senza di essa, l'esito del test è un'asserzione non verificabile da terzi, priva del requisito di non-repudiabilità che un audit trail tecnico richiede.

Il design originale dell'Evidence Store utilizzava un buffer a capacità fissa in memoria: i record più vecchi venivano rimossi silenziosamente quando il buffer si riempiva, generando riferimenti a record inesistenti nell'audit trail. Il risultato era un trail rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile aumentando la capacità: qualsiasi limite fisso avrebbe trasformato la capacità in un parametro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

L'Evidence Store scrive ogni record su file locali con flush immediato, senza buffering in memoria e senza dipendenze verso database o storage remoto: ogni record è permanente nel momento in cui viene prodotto e non esiste un limite superiore al numero di record per test. Il ciclo di vita dettagliato del componente è descritto nella Sezione 5.6.1. La scelta di operare esclusivamente su file locali è anche la condizione che abilita quanto descritto nella sottosezione seguente.

4.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

APIGuard Assurance non ha dipendenze di stato esterno a runtime: tutto lo stato mutabile del processo risiede in memoria o in file locali, e le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo e, opzionalmente, verso la Admin API del gateway. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool. Un tool che richiede database, message broker o cache esterna trasferisce la propria affidabilità a servizi esterni: l'exit code al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti quei servizi erano disponibili e corretti, una garanzia che il tool non può fornire autonomamente.

Nessun client di database, cache distribuita o Software Development Kit (SDK) di storage remoto figura tra le dipendenze obbligatorie del progetto: il deployment si riduce all'installazione del package e alla fornitura del file di configurazione. Questa assenza è anche la condizione che rende affidabile D6.P1: i semantic exit codes discussi nella Sezione 7.2 riflettono fedelmente lo stato dell'assessment perché nessun servizio accessorio può alterarli silenziosamente.

4.3 Domini di Sicurezza e Box Gradient Applicato

Le sezioni precedenti hanno definito le proprietà architetturali e strutturali su cui il sistema si fonda. Questa sezione definisce cosa il sistema misura e secondo quale logica

organizza le proprie misure: è il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 3, dove le categorie di vulnerabilità sono state analizzate in quanto problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

4.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea. Alcune garanzie sono verificabili senza nessuna conoscenza del sistema, interrogando semplicemente gli endpoint esposti senza credenziali; altre richiedono uno stato autenticato; altre ancora diventano accessibili solo attraverso il piano di configurazione amministrativo del gateway. La tassonomia adottata riconosce questa struttura di precondizioni, organizzando le garanzie in domini che rispecchiano categorie semantiche distinte di rischio.

La Tabella 4.2 li riporta con il loro titolo e l'ambito di verifica che ciascuno racchiude.

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control (RBAC), BOLA).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (Hash-based Message Authentication Code (HMAC)).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema. (<i>Milestone 2</i>)
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, SSRF, race condition su operazioni critiche.

Tabella 4.2: Domini di Assurance di APIGuard Assurance.

La sequenza dei primi tre domini rispecchia una catena logica di precondizioni: non ha senso verificare l'autorizzazione (D2) senza avere prima verificato che l'autenticazione funzioni (D1), così come non ha senso verificare l'autenticazione senza conoscere la superficie di attacco (D0). Un assessment che verifichi D2 in assenza di certezze su D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto. I domini D3-D7 non seguono la stessa dipendenza lineare: appartengono a famiglie semantiche distinte, ciascuna con le proprie precondizioni di accesso, senza vincoli di ordinamento reciproco tra di loro. Questa catena logica si riflette nelle dipendenze dichiarate dai singoli test che operano su questi domini, il cui meccanismo di risoluzione è descritto nella Sezione 4.2.2.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturalmente predisposto nella struttura del sistema, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

4.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing, nota nella letteratura come distinzione tra gradi di visibilità del sistema sotto test [rif. Sezione 2.X], viene derivata in APIGuard Assurance dalle precondizioni concrete che ogni garanzia impone al tester: non è una categoria assegnata retroattivamente ai test, ma il prodotto di ciò che ciascuna verifica richiede prima ancora di poter iniziare. Alcune garanzie sono fisicamente inaccessibili senza uno specifico livello di privilegio, e quella inaccessibilità è il criterio che determina il livello.

Il mapping adottato, formalizzato come proprietà D3.P3 e sintetizzato nella Tabella 4.3, correla la strategia operativa alle precondizioni concrete che ogni classe di garanzie impone al tester. Il livello di visibilità di ogni test è determinato da ciò che la verifica richiede per essere condotta correttamente.

I controlli perimetrali applicati dal gateway a qualsiasi interlocutore, autenticato o meno, non presuppongono nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo stesso indipendentemente da chi effettua la richiesta. Eseguirli senza credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test `BLACK_BOX` sono la scelta naturale per questa classe di garanzie.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno stato autenticato con almeno due identità di ruolo distinto. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.

Tabella 4.3: Box Gradient applicato in APIGuard Assurance (D3.P3).

risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. La strategia GREY_BOX è la risposta a questa preconditione.

I test WHITE_BOX occupano una posizione metodologicamente distinta. Alcune garanzie sono più efficacemente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione TLS supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test WHITE_BOX delegano la lettura della configurazione all'adapter del gateway descritto nella Sezione 4.2.4, anziché effettuare probe sull'interfaccia pubblica.

In contesti industriali, la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment. Un'organizzazione che non dispone di accesso amministrativo al gateway può eseguire l'intera suite BLACK_BOX e GREY_BOX, mentre i test WHITE_BOX transitano a SKIP con motivazione esplicita. La Sezione 7.2 descrive come questa variabilità di privilegio si integri con le pipeline CI/CD.

Capitolo 5

Implementazione

Le proprietà architetture formalizzate nel Capitolo 4 si manifestano in questa sezione nella loro implementazione concreta. Il grado di trattamento rispecchia il peso strutturale di ciascuna: alcune definiscono l'organizzazione di interi componenti e sono trattate in sezioni dedicate; altre derivano dalle proprietà stabilite nel Capitolo 4 e vengono richiamate nel contesto del componente in cui si realizzano; altre ancora, di natura prevalentemente operativa, trovano collocazione nelle sezioni di questo capitolo pertinenti al loro locus, o nei capitoli successivi dedicati alla validazione e all'integrazione con i processi di sviluppo. La Tabella 4.1 costituisce la mappa di navigazione verso le sezioni dove ciascuna proprietà è introdotta.

Il Capitolo 4 ha percorso il primo livello di questo approccio top-down: ha stabilito i principi fondamentali del sistema, la metodologia di assessment, i domini di sicurezza e le proprietà architetture che ne definiscono la struttura. Questo capitolo ne rappresenta il livello successivo nella discesa: i principi prendono forma in moduli Python, le proprietà si concretizzano in classi, meccanismi e scelte implementative specifiche. Dove un componente è stato introdotto nella sua motivazione architetture, le sezioni seguenti aggiungono il dettaglio implementativo con un rimando esplicito alla sezione di origine.

5.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La sequenza rispecchia una dipendenza logica reale tra le operazioni, dove ogni fase produce le precondizioni che la successiva presuppone; tale dipendenza determina anche la semantica degli errori. Le fasi 1, 2 e 4 producono la configurazione validata, la mappa degli endpoint e il grafo delle dipendenze tra test: senza questi tre elementi nessun test può essere eseguito in modo significativo, e un errore in una qualsiasi di esse rende l'in-

tero assessment privo di fondamento. Le fasi 5, 6 e 7 operano invece su un contesto già validato, dove il fallimento è sempre recuperabile localmente: un test fallito produce un `TestResult(ERROR)` e la pipeline prosegue; un teardown parziale viene registrato come `WARNING` senza invalidare i risultati che lo precedono.

Fase	Cartella	File
1. Inizializzazione	<code>config/</code>	<code>loader.py</code>
2. OpenAPI Discovery	<code>discovery/</code>	<code>openapi.py</code>
3. Costruzione Contesti	<code>src/</code>	<code>engine.py</code>
4. Test Discovery e DAG	<code>tests/</code> , <code>core/</code>	<code>registry.py</code> , <code>dag.py</code>
5. Esecuzione	<code>src/</code>	<code>engine.py</code>
6. Teardown	<code>core/</code>	<code>context.py</code>
7. Report	<code>report/</code>	<code>renderer.py</code>

Tabella 5.1: Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).

La Tabella 5.1 riporta il mapping tra fase e modulo responsabile: ogni fase corrisponde a un modulo separato nel filesystem (Figura 5.1), e `engine.py` è il solo file autorizzato a importare da tutti gli strati (Sezione 4.2.1). Il dettaglio di ogni fase viene presentato nell'elenco sottostante; si nota che le eccezioni citate per le fasi bloccanti appartengono alla gerarchia custom analizzata nella Sezione 5.1.2, dove il loro phase mapping e le relative politiche di recovery sono descritte.

1. **Fase 1: Inizializzazione** (Sezione 4.1.3). `config/loader.py` legge e valida `config.yaml` contro lo schema Pydantic, interpola i segreti da variabili d'ambiente e costruisce il modello `RuntimeConfig`.
 - *In caso di errore:* `ConfigurationError` interrompe lo startup.
 - *Perché bloccante:* senza configurazione valida non esistono credenziali, URL di target né parametri di esecuzione; qualsiasi fase successiva opererebbe su basi non definite.
2. **Fase 2: OpenAPI Discovery** (Sezione 4.1.2). `discovery/openapi.py` scarica la specifica OpenAPI del target, dereferenzia i `$ref`¹ tramite `prance`, la valida con `openapi-spec-validator` e costruisce l'`AttackSurface`: la mappa strutturata degli endpoint che ogni test interroga per determinare il proprio scope.
 - *In caso di errore:* `OpenAPILoadError` interrompe lo startup.

¹Le specifiche OpenAPI usano `$ref` per referenziare schemi riutilizzabili definiti altrove nel documento o in file esterni. La dereferenziazione sostituisce ogni `$ref` con il contenuto del componente referenziato, producendo una specifica autonoma e navigabile senza dipendenze esterne.

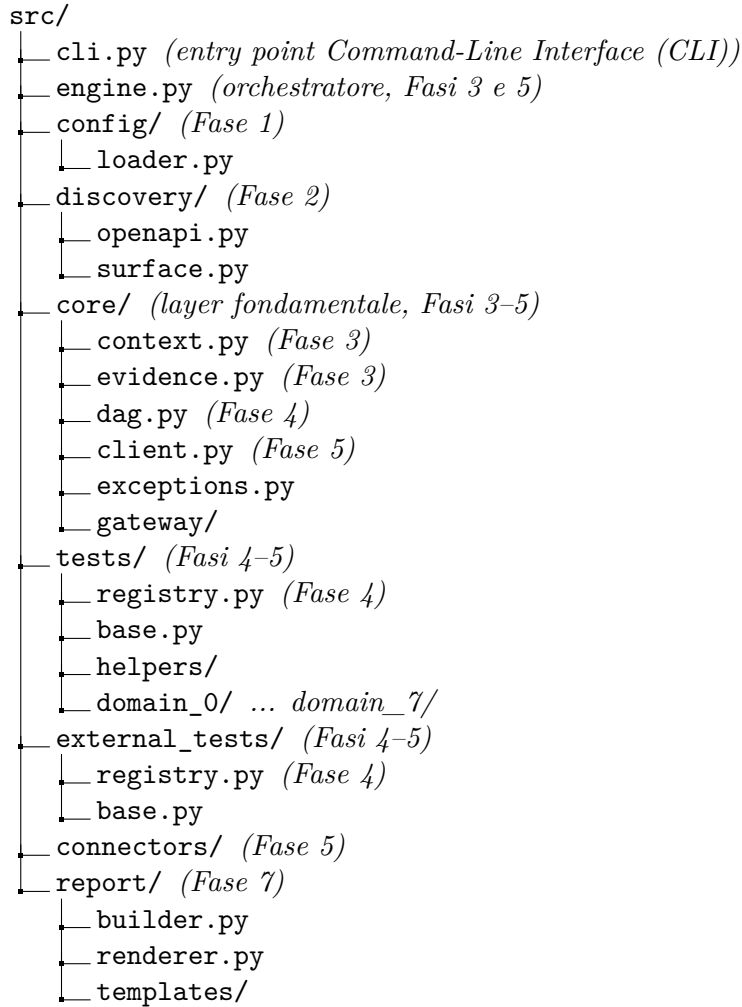


Figura 5.1: Layout di `src/` con mapping alle fasi della pipeline.

- *Perché bloccante:* la specifica OpenAPI è la sorgente esclusiva della conoscenza del target (Sezione 4.1.1); senza di essa nessun test può determinare su quali endpoint operare.
3. **Fase 3: Costruzione Contesti.** `engine.py` istanzia i tre oggetti di runtime condivisi: `TargetContext` (conoscenza del target frozen) e `TestContext` (stato dell'assessment mutabile), descritti nella Sezione 5.1.3, e `EvidenceStore` (streaming su disco), descritto nella Sezione 5.6.1.
- *In caso di errore:* nessuna eccezione prevista; la costruzione dei contesti non produce fallimenti recuperabili.

4. **Fase 4: Test Discovery e DAG.** `tests/registry.py` scopre dinamicamente i test disponibili via `pkgutil` e applica i filtri di proprietà e strategia configurati (Sezione 5.2.1); la lista risultante viene passata a `core/dag.py`, che costruisce il grafo delle dipendenze e calcola l'ordinamento topologico in batch (Sezione 5.2.2).
 - *In caso di errore:* `DAGCycleError` interrompe lo startup.
 - *Perché bloccante:* un ciclo nelle dipendenze rende il grafo non risolvibile; tentare l'esecuzione produrrebbe stallo o ordinamento non deterministico.
5. **Fase 5: Esecuzione.** `engine.py` itera sui `ScheduledBatch` prodotti dalla fase precedente ed esegue i test in ordine topologico, invocando `BaseTest.execute()` per i test nativi (Sezione 5.3) e `ExternalToolTest.execute()` per i test con tool esterni (Sezione 5.4); il meccanismo di isolamento degli errori è descritto nella Sezione 5.1.1.
 - *In caso di errore:* le eccezioni interne a ogni test sono catturate dal contratto di `execute()` e convertite in `TestResult(ERROR)`; la pipeline prosegue con il test successivo.
 - *Perché non-bloccante:* il fallimento di un singolo test è un dato dell'assessment, non un motivo per invalidare i risultati degli altri.
6. **Fase 6: Teardown.** `core/context.py` drena il registro delle risorse create durante l'esecuzione e tenta la loro rimozione in ordine LIFO tramite `SecurityClient`; il meccanismo è descritto nella Sezione 5.3.8.
 - *In caso di errore:* `TeardownError` registrato come `WARNING` strutturato; non invalida i risultati dell'assessment.
 - *Perché non-bloccante:* il mancato cleanup è un avvertimento operativo, non un'invalidazione scientifica dei finding già prodotti.
7. **Fase 7: Report.** `report/builder.py` aggrega i `TestResult` in un `ReportData` e produce tre output: `evidence.json` (archivio forense), `assessment_report.html` (report operativo con log completo delle transazioni) e `apiguard_report.json` (serializzazione strutturata per CI/CD), descritti nella Sezione 5.6.3.
 - *In caso di errore:* ogni sotto-operazione di 7 dispone di un proprio blocco di gestione indipendente; un fallimento su un output non impedisce la produzione degli altri.

Il ruolo di `engine.py` è limitato all'orchestrazione pura: esegue le fasi nell'ordine corretto, trasferisce i contesti da una fase all'altra, e non contiene logica di dominio di nessun tipo (né regole di sicurezza, né criteri di valutazione, né conoscenza del formato della specifica OpenAPI). La scelta ha una ragione strutturale: un orchestratore che incorpora logica di dominio diventa il punto di concentrazione delle dipendenze, rendendo difficile testare i componenti in isolamento e trasformando ogni sostituzione di fase in un'operazione ad alto rischio di regressione. La coerenza con il principio del flusso monodirezionale (Sezione 4.2.1) è verificabile per ispezione diretta: `engine.py` importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`.

5.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto tra l'orchestratore (`engine.py`) e i test individuali si caratterizza per un'assenza strutturale: nessun blocco `try/except` racchiude la chiamata a `test.execute()` nell'engine. Si tratta di una scelta deliberata, che delega la responsabilità della gestione delle eccezioni a livello di test, impedendo che una singola probe possa interrompere l'esecuzione delle altre per errori non previsti. Il perimetro di responsabilità dell'engine termina alla firma del contratto: invoca `test.execute()` e riceve un `TestResult`; qualsiasi condizione interna al test, normale o anomala, non attraversa quel confine.

Perché questo contratto funzioni, `BaseTest.execute()` deve garantire due invarianti: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except` `Exception` all'interno di `execute()`: in caso di eccezione imprevista, l'helper `_make_error()` costruisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri. Un test con un bug interno, una risposta HTTP inattesa o un timeout non gestito produce un `TestResult(ERROR)` registrato nell'evidenza con il proprio traceback; la pipeline prosegue con il test successivo nel batch senza interruzioni. La solidità complessiva del sistema non è condizionata dalla correttezza di nessun singolo test.

5.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti descritto in Tabella 5.1 è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool ereditano da `ToolBaseError`, radice comune della gerarchia. La struttura offre al codice chiamante due granularità di intercettazione: un `except ToolBaseError` copre qualsiasi condizione di errore del tool in un unico punto; scendere su un'eccezione specifica, come `except ConfigurationError` o `except DAGCycleError`, permette di distinguere la causa e rispondere in modo differenziato. Tale granularità rispecchia in modo diretto la distinzione tra errori bloccanti e non-bloccanti stabilita a livello architetturale, con-

cretizzandosi in 11 classi, distribuite su 3 file, riportate nella Tabella 5.2 con la relativa fase.

Classe	File	Comportamento
<code>ToolBaseError</code>	<code>exceptions.py</code>	Radice della gerarchia; non si solleva direttamente
<code>ConfigurationError</code>	<code>exceptions.py</code>	Fase 1: fatale, config non valida; blocca lo startup
<code>OpenAPILoadError</code>	<code>exceptions.py</code>	Fase 2: fatale, spec irraggiungibile o invalida; blocca lo startup
<code>DAGCycleError</code>	<code>exceptions.py</code>	Fase 4: fatale, ciclo nelle dipendenze; blocca lo startup
<code>AuthenticationSetupError</code>	<code>exceptions.py</code>	Helper auth: fatale se le credenziali configurate sono invalide
<code>SecurityClientError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>ExternalToolError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>TeardownError</code>	<code>exceptions.py</code>	Fase 6: recuperata, registrata come <code>WARNING</code>
<code>GatewayAdapterError</code>	<code>gateway/base.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>SeedGeneratorFetchError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10
<code>SeedGeneratorParseError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10

Tabella 5.2: Gerarchia delle eccezioni custom (Fasi 1–4 bloccanti; Fasi 5–6 recuperate localmente).

La prima scelta che riflette una decisione di design consapevole e non un vincolo tecnico, riguarda la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l'eccezione è definita nello stesso file dell'interfaccia che la solleva, rendendo il contratto dell'ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

A questa si affianca una seconda scelta altrettanto deliberata: l'assenza di `ExternalToolNotFoundError`. Un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa attesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector, come descritto nella Sezione 5.4.2. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un'eccezione. Introdurre

`ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

5.1.3 Costruzione dei Contesti di Esecuzione (D1.P3)

In Phase 3, l'engine assembla i due oggetti che attraversano l'intera pipeline: `TargetContext` e `TestContext`. La motivazione architetturale della loro separazione è discussa nella Sezione 4.2.3.

`TargetContext` è dichiarato `frozen=True` a livello di modello Pydantic: qualsiasi tentativo di scrittura dopo la costruzione produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che il sistema conosce del target prima che la pipeline inizi: l'`AttackSurface` costruita dalla specifica OpenAPI in Phase 2, le credenziali risolte dal loader, i parametri di tuning per singolo test, la configurazione dei tool esterni e l'adapter del gateway iniettato se configurato. Ogni test che legge `TargetContext` ha la garanzia assoluta che i dati che legge sono identici a quelli letti da ogni altro test nella stessa run.

`TestContext` è mutabile, ma l'accesso non avviene tramite un dizionario libero: lo stato è organizzato in tre canali tipizzati con responsabilità distinte. Il canale token gestisce le credenziali acquisite durante la run tramite `set_token/get_token/has_token`, con chiavi che sono costanti nominate (`ROLE_ADMIN`, `ROLE_USER_A`, `ROLE_USER_B`) e non stringhe arbitrarie. Il canale teardown mantiene il registro LIFO delle risorse create, descritto nella Sezione 5.3.8. Il canale dati condivisi permette a test in batch successivi di consumare risultati prodotti da batch precedenti tramite `set_shared/get_shared`, con la convenzione `"{test_id}.{nome_dato}"` che rende esplicito quale test ha prodotto ogni valore.

5.2 Discovery Dinamico e Risoluzione del Grafo

Come stabilito nella Sezione 4.2.2 e anticipato nella Fase 4 dell'elenco nella Sezione 5.1, il DAG delle dipendenze tra test non è risolto dall'engine direttamente, ma delegato a due componenti con responsabilità distinte: il `TestRegistry`, che determina a runtime quali test esistono per ispezione del filesystem, e il `DAGScheduler`, che riceve la lista dei test scoperti e calcola l'ordine topologico a partire dalle dipendenze dichiarate. Il registry opera su moduli e classi Python; lo scheduler opera su grafi di stringhe. La separazione riflette la differenza di dominio tra le due operazioni: una riguarda la composizione del set di test attivi, l'altra la loro sequenza di esecuzione.

5.2.1 Discovery Dinamico via pkgutil (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` scopre le classi concrete di test a runtime attraverso tre fasi sequenziali, denominate R1, R2 e R3, senza fare ricorso ad alcun elenco di registrazione manuale:

1. **Phase R1: Scansione del package.** La scansione del package `src.tests` è affidata a `pkgutil.walk_packages()`, che ne attraversa l'intera struttura importando tutti i moduli il cui nome finale rispetta il prefisso `test_`. Il meccanismo è indipendente dall'organizzazione delle sottodirectory: qualsiasi directory di `src/tests/` configurata come package Python con il proprio `__init__.py` viene inclusa automaticamente, a prescindere dal nome. Per convenzione, i test sono organizzati in directory `domain_N/`, una per ciascun dominio di assurance, ma il registry non vincola questa struttura. Ogni file di test adotta la convenzione `test_N_M_description.py`, dove `N` è il numero di dominio, `M` il numero progressivo del test all'interno del dominio e `description` una label in `snake_case`; il `test_id` dichiarato nella `ClassVar` segue il formato puntato corrispondente `N.M`. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.
2. **Phase R2: Estrazione delle classi concrete.** Dai moduli importati vengono estratte le sottoclassi concrete di `BaseTest` tramite `inspect.getmembers()`, scaricando la classe base stessa, le sottoclassi astratte con metodi non implementati, e le classi prive dei `ClassVar` obbligatori verificati da `has_required_metadata()`.
3. **Phase R3: Filtraggio per priorità e strategia.** Vengono trattenuti i test la cui `priority` non supera la soglia `min_priority` e la cui `strategy` è inclusa nell'insieme delle strategie abilitate in `config.yaml`. Il filtraggio avviene per classe, prima dell'istanziamento, e ogni test escluso viene registrato a livello `DEBUG` con il motivo dell'esclusione.

Il meccanismo ha una conseguenza diretta sull'estensibilità del sistema: aggiungere un test richiede la creazione di un file nella directory di dominio corretta rispettando la convenzione `test_N_M_description.py` e implementare il contratto di `BaseTest` con i `ClassVar` obbligatori; nessun altro file deve essere modificato, nessuna registrazione manuale è necessaria.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue le stesse fasi R1-R3, adattate alla gerarchia `ExternalToolTest` e alla convenzione di nomenclatura `ext_test_*.py`, con una fase aggiuntiva denominata R4. Dopo il filtraggio, il registry raggruppa i test sopravvissuti per `tool_name` e chiama `is_available()` una sola volta per gruppo: se il tool è presente, istanzia un connector condiviso e lo inietta in tutti i test del gruppo tramite `_injected_connector`; se il tool è assente, imposta

`_skip_reason_from_registry` su ogni test del gruppo, in modo che `execute()` restituisca `SKIP` senza tentare l'invocazione. Un solo `WARNING` viene emesso per tool mancante, indipendentemente da quanti test ne dipendono.

Lo stesso principio di estensibilità vale per i test esterni: aggiungere un nuovo `ExternalToolTest` richiede la creazione di un file rispettando la nomenclatura `ext_test_*.py`, implementare il contratto di `ExternalToolTest` con i `ClassVar` obbligatori e dichiarare il `tool_name` corrispondente al connector. Questa proprietà costituisce il prerequisito tecnico del plugin system delineato come traiettoria futura nella Sezione 7.3: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni di nomenclatura viene rilevato automaticamente al successivo avvio della pipeline, senza modifiche al core.

5.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il `DAGScheduler` in `src/core/dag.py` riceve la lista dei `test_id` prodotta dal registry (Sezione 5.2.1) e restituisce una sequenza ordinata di `ScheduledBatch`: ciascun batch raggruppa i test che possono essere eseguiti nello stesso livello topologico perché le loro dipendenze sono già state soddisfatte nei batch precedenti. Il meccanismo di risoluzione, introdotto a livello architetturale nella Sezione 4.2.2, si concretizza qui nel pattern `get_ready() / done()` di `graphlib.TopologicalSorter`, la libreria standard Python introdotta nella versione 3.9. Il `DAGScheduler` costruisce, a partire dal campo `depends_on` di ogni test, una mappa da ogni `test_id` all'insieme dei suoi predecessori diretti, e la passa a `TopologicalSorter` come input per la risoluzione del grafo.² Chiama poi `prepare()` per intercettare eventuali cicli prima dell'esecuzione, e drena l'ordinamento livello per livello: ogni chiamata a `get_ready()` restituisce i nodi pronti, `done(*ready_nodes)` sblocca quelli che da essi dipendono, e ogni livello produce uno `ScheduledBatch`.³

`ScheduledBatch` è un `dataclass(frozen=True)` con due campi: `batch_index` (intero che indica il livello topologico incrementando a partire da zero) e `test_ids` (lista ordinata di stringhe). L'immutabilità garantisce che l'engine non possa alterare un batch durante l'esecuzione. Il determinismo all'interno di ogni batch, requisito discusso nella Sezione 4.1.4, dipende da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib.TopologicalSorter.get_ready()` restituisce i nodi pronti senza garantire nessun ordine sull'insieme estratto: a parità di grafo e di dipendenze dichiarate, l'ordine di iterazione può variare tra run, versioni di Python o piattaforme. L'ordinamento lessicografico esplicito sul `test_id` è l'unica fonte di ordine deterministico all'interno di un

²Ogni test dichiara solo le dipendenze immediate: se il test A dipende da B e B dipende da C, è sufficiente che A dichiari `depends_on = ["B"]`. Il vincolo su C è già catturato dalla dipendenza di B su C, e `TopologicalSorter` lo propaga automaticamente nel calcolo dell'ordinamento.

³Passare un nodo a `done()` segnala al sorter che quel `test_id` è stato completato: tutti i test che lo dichiarano in `depends_on` vedono soddisfatto quel vincolo e diventano eleggibili per la successiva chiamata a `get_ready()`.

batch: due run con le stesse dipendenze dichiarate producono sempre la stessa sequenza di esecuzione, condizione necessaria perché D3.P4 sia verificabile empiricamente e non solo enunciabile.

Un caso limite, documentato nella Sezione 4.2.2 a livello architetturale, è gestito dalla stall detection: `is_active()` restituisce `True` ma `get_ready()` non produce nodi, segnalando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. Il `DAGScheduler` calcola in questo caso l'insieme dei `test_id` mai estratti, li registra in un `ERROR` strutturato con il campo `unscheduled_test_ids`, e interrompe il drain. Il log fornisce all'operatore l'elenco esatto dei test coinvolti senza richiedere la lettura di un traceback generico.

5.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Il loro ciclo di vita nella pipeline, dalla scoperta in Fase 4 all'invocazione in Fase 5, è descritto nella Sezione 5.1; questa sezione ne descrive la struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, le invarianti sul risultato che il sistema applica per costruzione, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, la policy con cui le probe HTTP vengono interpretate, il meccanismo di risoluzione dei path parametrici e il catalogo dei payload di attacco, e il meccanismo di cleanup delle risorse create durante la verifica. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

5.3.1 Gerarchia Duale `BaseTest` / `ExternalToolTest` (D1.P4)

Il sistema ospita due categorie di test con contratti distinti: i test nativi estendono `BaseTest` in `src/tests/base.py`; i test con tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele e indipendenti, per ragioni tecniche precise che la struttura del sistema rende evidenti.

La scelta di mantenere le due gerarchie separate risponde a un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target applicativo. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta di dati al proprio connector, che invoca un processo di sistema o una libreria Python. Fare ereditare `ExternalToolTest` da `BaseTest` avrebbe reso disponibile a ogni

test esterno un'interfaccia pensata per un pattern di integrazione diverso, ampliando la superficie di errore e oscurando il contratto che il test deve effettivamente rispettare.

Dal punto di vista dell'engine, la distinzione è quasi invisibile: entrambe le gerarchie producono un `TestResult`, vengono scoperte dal registry con gli stessi meccanismi di Phase R1-R3 descritti nella Sezione 5.2.1 (con l'aggiunta di Phase R4 per i test esterni), e vengono schedate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]` propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono riportati con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto che `BaseTest` impone a ogni implementazione concreta si articola in due livelli:

- **Dichiarazione statica via `ClassVar`:** ogni test deve dichiarare `test_id` (identificatore univoco corrispondente all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (`BLACK_BOX`, `GREY_BOX`, `WHITE_BOX`), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). `has_required_metadata()` scarta le classi con `ClassVar` incompleti prima che vengano istanziate, così le implementazioni parziali non raggiungono mai la pipeline.
- **Implementazione di `execute(target, ctx, store)`:** il metodo che garantisce le due invarianti discusse nella Sezione 5.1.1: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'engine.

5.3.2 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. La coerenza tra lo stato di un `TestResult` e il suo contenuto, se affidata alla disciplina distribuita di implementazioni indipendenti, è una proprietà che il sistema non può garantire strutturalmente: uno stato `FAIL` senza evidenza, uno `SKIP` senza motivazione, un `PASS` con findings allegati sono tutti costruibili sintatticamente senza che il sistema se ne accorga, producendo entry di report formalmente validi ma privi di significato verificabile. La soluzione adottata sposta l'enforcement a livello di tipo, tramite un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti nel momento in cui ogni test costruisce il proprio risultato, ovvero alla chiamata di `_make_pass()`, `_make_fail()` o `_make_skip()`, prima che l'oggetto raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un FAIL senza evidenza non è un FAIL: è un'asserzione.
2. `status=SKIP` richiede `skip_reason` non `None`. Ogni SKIP deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un PASS non può coesistere con violazioni dichiarate.

La centralizzazione in un singolo `model_validator` ha un effetto architetturale preciso: il contratto tra stato e contenuto del `TestResult` è enforced nel punto in cui l'oggetto viene costruito, indipendentemente da quale test lo costruisce e da quale helper utilizza. Una violazione solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()` o `_make_skip()` costruisce l'oggetto, con un traceback che punta direttamente alla riga incriminata nel test responsabile. Ogni nuova implementazione di test ottiene la verifica automaticamente: il validator intercetta risultati inconsistenti nel punto stesso della costruzione, senza che il test debba contenere logica di validazione propria.

5.3.3 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test `GREY_BOX` non possono iniziare la propria probe senza token validi: il gateway respinge qualsiasi richiesta priva di credenziali prima che la logica di autorizzazione venga anche solo valutata. Con una parte significativa dei 18 test attivi in Milestone 1 in modalità `GREY_BOX`, distribuire la logica di login in ogni test produrrebbe autenticazioni ripetute verso lo stesso target per ruoli già acquisiti. `acquire_tokens()` in `src/tests/helpers/auth.py` risolve questo problema centralizzando la gestione dei token nel `TestContext`: alla chiamata, la funzione verifica prima se il `TestContext` contiene già un token valido per il ruolo richiesto tramite `ctx.has_token(role)`; solo se il token è assente esegue il login HTTP e lo scrive nel canale token tramite `ctx.set_token(role, token)`. Il numero di login effettivi verso il target è così esattamente uguale al numero di ruoli distinti configurati, indipendentemente da quanti test li richiedano.

Il dispatcher in `auth.py` legge il campo `auth_type` dal `config.yaml` e delega l'acquisizione dei token all'implementazione corrispondente, astraendo i test dal dettaglio del protocollo. Nella Milestone 1 sono supportati due metodi: `forgejo_token`, che acquisisce un token tramite l'endpoint Forgejo-specifico `POST /api/v1/users/{username}/tokens` con HTTP Basic Auth, e `jwt_login`, un flusso generico configurabile tramite `login_endpoint`, `username_body_field` e `token_response_path`, adatto a qualsiasi target che esponga un endpoint di login JSON, da crAPI a Django REST Framework. Aggiungere il supporto per un nuovo meccanismo richiede una nuova implementazione del flusso e un ramo

aggiuntivo nel dispatcher; i test rimangono invariati perché il loro contratto con il layer di autenticazione è espresso esclusivamente in termini di ruolo richiesto.

5.3.4 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` per la classe di vulnerabilità nel Common Weakness Enumeration (CWE), e `tags` per le categorie OWASP, i riferimenti National Institute of Standards and Technology (NIST) e i documenti tecnici Request for Comments (RFC). Entrambi vengono propagati fino al modello `Finding` tramite il campo `references: list[str]` e riportati dal report HTML a fianco di ogni risultato: un analista può risalire direttamente dal finding al codice CWE, alla categoria OWASP API Security e alle sezioni normative pertinenti senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un `Finding` è evidenza di una violazione: appartiene esclusivamente ai risultati `FAIL`, il suo contenuto deve essere sufficiente a giustificare la valutazione negativa, e viene conteggiato nei totali aggregati del report. Un `InfoNote` è un'annotazione informativa allegabile a un risultato `PASS`: documenta contesto rilevante per l'analista senza alterare il verdetto né influenzare i contatori. La distinzione è architetturale: i contatori del report aggregano esclusivamente oggetti `Finding`, rendendo il totale affidabile e non influenzabile da osservazioni di contorno. La coerenza tra tipo di risultato e tipo di annotazione è enforced come descritto nella Sezione 5.3.2: il `model_validator` di `TestResult` garantisce che un `PASS` non possa coesistere con `Finding` nel momento stesso della costruzione dell'oggetto.

Il valore operativo della distinzione emerge con chiarezza nel test 4.3, che verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone healthcheck passivi che svolgono una funzione parzialmente equivalente. Il test registra `PASS` perché il requisito minimo è soddisfatto, e allega una `InfoNote` che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso `FAIL` né un `PASS` silenzioso che nasconde la sfumatura.

5.3.5 Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)

Ogni test nativo che interroga un endpoint del target riceve una risposta HTTP che il tool deve interpretare e tradurre in un verdetto preciso. Il problema non è banale: uno status 404 Not Found su un endpoint parametrico come `/users/{id}/repos` può

significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un **404** come **FAIL** in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un framework a tre esiti applicato a ogni probe HTTP nei test nativi: ogni risposta è classificata come **ENFORCED** (il controllo funziona), **BYPASSED** (il controllo è assente o mal configurato) o **INCONCLUSIVE** (il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**). Il mapping specifico degli status code a ciascun esito dipende dalla natura del test: per i test **BLACK_BOX** che verificano l'enforcement dell'autenticazione, la forma è la seguente:

1. **ENFORCED**: la risposta è 401 o 403. Il gateway ha applicato un controllo di accesso sul chiamante non autenticato.
2. **BYPASSED**: la risposta è 2xx. Il gateway non ha richiesto autenticazione. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding** ⁴.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

La codifica a tre esiti presuppone che la risposta ricevuta rifletta il comportamento del controllo di accesso, non un effetto collaterale della probe stessa. Questa condizione regge per i path fissi della specifica, dove il test raggiunge una risorsa che esiste e la risposta è interpretabile. Per i path parametrici la situazione è più sottile: un endpoint come `/users/{id}/repos` ha una struttura valida, ma il valore concreto del parametro determina se quella specifica risorsa esiste sul target. Con un valore che non corrisponde a nessun oggetto reale, il gateway o il backend restituisce **404** perché la risorsa è assente, non perché stia applicando un controllo di accesso; il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. Classificare quel **404** come **INCONCLUSIVE** è l'unica risposta metodologicamente corretta. Da questa asimmetria nasce la segregazione in due tier: il Tier A raccoglie i path fissi e quelli parametrici per cui il seed è configurato (Sezione 5.3.6), dove la probe usa valori concreti verificati sul target e la risposta è interpretabile; il Tier B raccoglie i path parametrici senza seed, dove il risultato

⁴Esempio osservato su Forgejo: senza path seed, il probe su `/api/v1/repos/{owner}/{repo}` usa il placeholder "1" per entrambi i parametri; Forgejo risponde **404** perché la risorsa non esiste, la richiesta non raggiunge il middleware di autenticazione e il risultato è **INCONCLUSIVE**. Con seed configurato, la stessa richiesta produce **ENFORCED** o **BYPASSED**.

è INCONCLUSIVE e l'evidenza segnala all'operatore che configurare il seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. Le probe con metodo DELETE su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta DELETE con un path valido potrebbe eliminare risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint DELETE parametrico, il probe viene eseguito con il valore letterale "apiguard-probe" come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

5.3.6 Path Seed System (D2.P7)

Il path seed system affronta una tensione strutturale dell'agnosticismo applicativo. Derivare tutta la conoscenza del target dalla specifica OpenAPI significa conoscere la topologia degli endpoint ma non i valori concreti dei loro parametri: `/users/{owner}/repos/{repo}` è un endpoint documentato, ma senza sapere quale `owner` e quale `repo` esistano realmente sul target, la probe usa un placeholder il cui path quasi certamente non esiste, producendo 404 e un risultato INCONCLUSIVE. Il seed risolve questa tensione fornendo per ogni parametro la possibilità di specificare un valore concreto verificato sul target⁵: la generazione tramite il comando CLI `generate-seed`, che interroga la specifica OpenAPI, produce un template da completare e inserire in `config.yaml`. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: ENFORCED se il gateway applica il controllo atteso, BYPASSED se non lo applica. La Sezione 6.3 riporta i dati quantitativi di questo effetto sull'assessment descritto in questo capitolo.

5.3.7 Catalogo dei Payload di Attacco (D2.P8)

I test nativi applicano il principio di separazione tra dati e logica di valutazione che, come verrà illustrato nella Sezione 5.4.1, governa anche il contratto dei connector: i dati di input alla verifica risiedono separati dalla logica che li interpreta. Nel caso dei test nativi, questo si concretizza nel catalogo dei payload. Le sequenze di URL usate dal test SSRF, gli header malformati per i test di input validation, e altri vettori di attacco risiedono in moduli puri in `src/tests/data/`, distinti dalla logica che li applica. Aggiungere un nuovo vettore richiede la modifica del catalogo, senza toccare il test che lo utilizza. La separazione ha una conseguenza operativa concreta in contesti enterprise,

⁵Un valore verificato è un identificatore di una risorsa realmente esistente nel deployment; nel caso di Forgejo: un utente, un repository, e così via.

dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo.

5.3.8 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il cleanup delle risorse si basa sulla registrazione esplicita al momento della creazione: ogni volta che un test crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_t(path, headers)` passando l'endpoint di cancellazione. La coppia creazione-registrazione avviene nella stessa transazione logica; se il test va in eccezione tra le due operazioni, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto: il meccanismo di isolamento degli errori descritto nella Sezione 5.1.1 non può recuperare uno stato che non è mai stato registrato.

In Phase 6, l'engine chiama `ctx.drain_resources()`, che restituisce le risorse in ordine LIFO. Tale ordine inverso rispetto alla creazione è necessario per gestire correttamente le dipendenze tra risorse: se un test ha creato prima un repository e poi una issue al suo interno, la issue dipende dal repository; cancellarli nell'ordine di creazione eliminerebbe prima il repository, rendendo impossibile la cancellazione della issue. Invertendo l'ordine, la issue viene rimossa prima che il repository smetta di esistere. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite `SecurityClient`; un fallimento solleva `TeardownError`, catturato e registrato come `WARNING` strutturato. Il fallimento del teardown è un avvertimento operativo, non una condizione che invalida i risultati prodotti nelle fasi precedenti. La Sezione 6.3 riporta i dati della verifica empirica, che ha rilevato zero risorse residue su due run indipendenti.

5.4 Gerarchia dei Connettori e Integrazione Tool Esterni

La controparte della gerarchia duale introdotta nella Sezione 5.3.1 è `ExternalToolTest`: i test che delegano la raccolta di dati a tool specializzati invece di comunicare direttamente con il target tramite `SecurityClient`. Seguono lo stesso ciclo di vita nella pipeline (scoperti in Fase 4 dall'`ExternalTestRegistry` con l'aggiunta di Phase R4, invocati in Fase 5), ma con un pattern di integrazione strutturalmente diverso. Scanner di vulnerabilità, analizzatori TLS, fuzzer producono tipi di evidenza che un client HTTP generico non può raccogliere: la negoziazione TLS richiede dialogo a livello di protocollo, l'analisi

con template nuclei richiede una base di signature aggiornata. Il layer dei connector è il punto in cui questa delega prende forma concreta: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune e restituisce al test un oggetto strutturato invece del raw output del processo. In assenza di questo layer, ogni test esterno conterrebbe sia la logica di verifica della garanzia sia la meccanica di subprocess management o di importazione della libreria, accoppiando due responsabilità che evolvono per ragioni distinte.

5.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

La gerarchia dei connector si articola su tre livelli, ciascuno dei quali eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

`BaseConnector` in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo `ClassVar` (`TOOL_NAME: str`) e tre metodi astratti che costituiscono il contratto universale verso il test. `is_available()` verifica se il tool è utilizzabile nell'ambiente corrente; `get_version()` restituisce la versione del tool per fini di audit; `run(target_url, timeout_seconds)` esegue il tool e restituisce un `ConnectorResult`. Il parametro `timeout_seconds` è privo di valore di default per design: ogni chiamante deve fornirlo esplicitamente dal `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca a seconda della natura del tool:

- `BaseSubprocessConnector` serve i tool invocati come processo di sistema (`NucleiConnector`, `TestsslConnector`): fornisce implementazioni concrete di `is_available()` e `get_version()` basate su subprocess, e il metodo `_run_subprocess()` che gestisce esecuzione, timeout e parsing dell'output. La ricerca del binario avviene su tre canali, fermandosi alla primo soddisfatto, nel seguente ordine: binario locale nella directory `tools/` del progetto, poi `shutil.which()` sul PATH di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. Il medesimo connector funziona così senza modifiche in sviluppo locale, in CI/CD e in ambienti containerizzati.
- `BaseLibraryConnector` serve i tool Python-native (`SslyzeConnector`): usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun subprocess, e legge la versione tramite `importlib.metadata`. La disponibilità è determinata dalla presenza del package nell'ambiente Python, non da un binario sul filesystem.

Le implementazioni concrete si limitano a dichiarare le `ClassVar` specifiche del tool⁶ e implementare `run()`, il cui contratto formalizza una separazione precisa tra raccolta di dati

⁶Per `BaseSubprocessConnector`: `TOOL_NAME`, `BINARY_NAME`, `LOCAL_TOOLS_SUBDIR` e `SERVICE_ENV_VAR`.

e valutazione della garanzia. Il connector restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta `tool_name`, `tool_version`, `raw_output: dict[str, Any]`, `exit_code`, `execution_time_ms` e `timed_out`: dati grezzi, privi di qualsiasi giudizio sulla sicurezza del target.

La valutazione è responsabilità esclusiva dell'`ExternalToolTest`, che implementa `_evaluate()` e applica il proprio oracle al `ConnectorResult`. La ragione di questa separazione è la riusabilità: il `NucleiConnector` non sa nulla della garanzia che il test vuole verificare, e per questo può essere condiviso da test con logiche di valutazione completamente diverse. Un secondo test che vuole usare nuclei per injection scanning scriverebbe un `_evaluate()` diverso ma userebbe lo stesso connector, senza alcuna modifica alla logica di invocazione.

5.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

Come descritto nella Sezione 5.2.1, la Phase R4 dell'`ExternalTestRegistry` raggruppa i test per `tool_name` e chiama `is_available()` una sola volta per gruppo. Il risultato di quella chiamata determina il lifecycle del connector per tutti i test del gruppo: disponibilità o assenza del tool si traduce in due comportamenti distinti, gestiti tramite due attributi di istanza di `ExternalToolTest`.

Se il tool è disponibile, il registry istanzia un connector condiviso e lo scrive in `_injected_connector` su ogni test del gruppo: tutti condividono la stessa istanza, costruita una volta sola. Se il tool è assente, il registry scrive invece in `_skip_reason_from_registry` la motivazione di skip già formulata. Al momento dell'esecuzione, `_run()` controlla questo campo come prima operazione: se valorizzato, restituisce `SKIP` immediatamente senza tentare la costruzione del connector né interrogare il filesystem. In entrambi i casi, la decisione è presa una volta sola in Phase R4 e propagata a tutti i test del gruppo tramite questi due attributi.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipendono da nuclei e il binario assente, il log produce un solo `WARNING` del tipo *"nuclei not found: 5 tests will SKIP"*, invece di cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante. Il comportamento complessivo del sistema in presenza di infrastruttura parzialmente disponibile, compresi gli scenari con Admin API assente o master switch disattivato, è descritto nella Sezione 5.5.2.

Per `BaseLibraryConnector`: `TOOL_NAME` e `LIBRARY_NAME`.

5.4.3 Classificazione A/B dei Connector (D2.P6)

I connector sono classificati in due categorie che riflettono il loro rapporto con la copertura nativa. I connector di Categoria A coprono garanzie che i test nativi non possono raggiungere con soli probe HTTP: la loro assenza produce **SKIP** su verifiche altrimenti inaccessibili. I connector di Categoria B estendono la copertura su garanzie già verificate parzialmente dai test nativi, aggiungendo profondità senza essere necessari per l'assessment di base. Nella Milestone 1, `NucleiConnector` e `TestsslConnector/SslyzeConnector` appartengono entrambi alla Categoria A: il primo perché l'analisi di Shadow API tramite template specializzati non è replicabile con probe HTTP generici; i secondi perché la negoziazione TLS richiede dialogo a livello di protocollo che supera le capacità di un client HTTP standard.

5.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Le dipendenze dichiarate in `[project.dependencies]` di `pyproject.toml` vengono installate automaticamente con qualsiasi `pip install apiguard-assurance`; includervi `sslyze` propagherebbe i suoi obblighi copyleft a chiunque distribuisca o utilizzi il tool, indipendentemente dall'uso effettivo della libreria. La sezione `[project.optional-dependencies]` del `pyproject.toml` è pensata esattamente per questo tipo di situazione: raccoglie le dipendenze con vincoli di licenza o peso computazionale che non devono essere imposti per default, lasciando all'operatore la scelta consapevole di installarle tramite un extra esplicito.

`sslyze` è dichiarata in `[project.optional-dependencies.sslyze]`. L'installazione esplicita tramite `pip install "apiguard-assurance[sslyze]"` abilita `ext.1.5.sslyze` e comporta l'accettazione consapevole della licenza AGPL per quella componente. In assenza dell'installazione, `SslyzeConnector.is_available()` restituisce `False` tramite `importlib.util.find_spec()`, il test riceve **SKIP** con motivazione esplicita, e il core rimane privo di qualsiasi import condizionale della libreria, isolandola strutturalmente. Il pattern è replicabile per qualsiasi dipendenza con vincoli analoghi configurando il `pyproject.toml` appositamente.

5.5 Implementazione del Gateway Adapter

La Sezione 4.2.4 ha stabilito il principio di agnosticismo verso il gateway e introdotto `BaseGatewayAdapter` come interfaccia astratta di sola lettura verso il piano di configurazione amministrativo. Questa sezione ne descrive la concretizzazione: l'implementa-

zione Kong che viene iniettata nel `TargetContext` in Fase 3 e i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili.

5.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

`KongGatewayAdapter` in `src/core/gateway/kong.py` è l'unica implementazione concreta di `BaseGatewayAdapter` nella Milestone 1, e realizza il principio dichiarato nella Sezione 4.2.4: i test `WHITE_BOX` 3.3, 4.2, 4.3 e 6.4 chiamano `target.gateway.get_plugins()`, `get_services()` e `get_upstreams()` tramite l'interfaccia astratta, senza contenere nessun riferimento a Kong. L'implementazione interroga la Admin API di Kong DB-less tramite richieste HTTP GET. Una scelta implementativa merita particolare attenzione: il componente usa `httpx` direttamente invece di `SecurityClient`. Le chiamate alla Admin API sono audit di configurazione dell'infrastruttura, non traffico di test verso il target applicativo; appartengono a un confine di fiducia distinto, non devono apparire nell'`EvidenceStore` e non sono soggette alla policy di retry del `SecurityClient`. Un client `httpx` dedicato mantiene i due confini separati nel codice in modo esplicito e verificabile.

I metodi pubblici dell'adapter, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()` e `get_plugin_by_name()`, delegano tutti la lettura a `_fetch_paginated()`. La Admin API di Kong restituisce le collezioni in formato paginato: ogni risposta contiene un campo `"data"` con gli elementi della pagina corrente e un campo `"next"` con il path della pagina successiva, oppure `null` se non ci sono ulteriori elementi. `_fetch_paginated()` segue questo cursore iterativamente finché `"next"` è `null`, concatenando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti entrano in una singola pagina e `"next"` è sempre `null`, quindi il ciclo termina alla prima iterazione; lo stesso meccanismo funziona correttamente in ambienti con Kong con database, dove le risorse possono essere distribuite su più pagine.

Il ciclo di vita dell'adapter si articola su due momenti distinti. In Fase 3, `check_connectivity()` verifica che la Admin API sia raggiungibile prima di popolare `TargetContext.gateway`: se la verifica fallisce, il campo rimane `None` e tutti i test `WHITE_BOX` transitano a `SKIP` per l'intera run. Durante l'esecuzione dei test (Fase 5), qualsiasi errore nelle chiamate HTTP verso l'Admin API viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

5.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta del sistema

non è un errore a cascata, ma una degradazione controllata su tre livelli distinti che produce risultati parziali corretti con motivazioni esplicite in ogni caso.

- **Livello 1 - master switch globale.** Il flag `external_tools.enabled` in `config.yaml` disabilita l'intera gerarchia `ExternalToolTest` in un'unica operazione: l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli, con zero overhead e zero SKIP nel log. Serve per ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.
- **Livello 2 - tool esterno assente.** Quando il master switch è attivo ma un singolo tool non è disponibile, la Phase R4 descritta nella Sezione 5.2.1 imposta `_skip_reason_from_registry` su ogni test del gruppo: ogni test riceve SKIP con motivazione esplicita, e un solo WARNING viene emesso nel log per tool mancante.
- **Livello 3 - Admin API del gateway assente.** Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce in Phase 3, `TargetContext.gateway` rimane `None`. Ogni test `WHITE_BOX` chiama `_requires_admin_api(target)` come prima operazione e restituisce SKIP con testo esplicito senza eseguire alcuna logica di verifica. L'assessment copre in questo scenario i soli domini `BLACK_BOX` e `GREY_BOX`, con un numero di SKIP pari al numero di test `WHITE_BOX` nel set attivo.

In tutti e tre i casi, gli SKIP prodotti sono distinguibili nel report per motivazione (campo `skip_reason`), permettendo all'operatore di separare le garanzie non verificate per impossibilità tecnica da quelle non verificate per prerequisiti mancanti.

5.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate dai test nativi e dai dati grezzi prodotti dai tool esterni. L'evidenza è ciò che distingue un risultato verificabile da un'asserzione: senza di essa, un FAIL non può essere riprodotto né contestato. Raccoglierla introduce però una responsabilità: un audit trail non sanitizzato che contiene token o credenziali è esso stesso un rischio di sicurezza: il file viene condiviso con il team di analisi e archiviato nei sistemi di ticketing, dove la sua esposizione sfugge al controllo di chi ha eseguito l'assessment. Questa sezione descrive come l'`EvidenceStore` gestisce raccolta affidabile e sanitizzazione strutturale durante la Fase 5, e come vengono generati i tre deliverable finali dell'assessment in Fase 7 (introdotti nella Sezione 5.1): `evidence.json` per l'archiviazione forense, `assessment_report.html` per la consultazione operativa e `apiguard_report.json` per l'integrazione strutturata nei sistemi CI/CD.

5.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

Il design dell'`EvidenceStore` è motivato nella Sezione 4.2.5, dove viene discusso il problema strutturale del buffer a capacità fissa che produceva audit trail rotti in silenzio. Questa sezione descrive l'implementazione concreta che ha risolto quel problema.

Il ciclo di vita dell'`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell'engine. In Phase 3, l'engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence_tmp/`. Prima di eseguire ogni test in Phase 5, l'engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl` in modalità append e lo mantiene aperto per l'intera durata del test. Durante l'esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un singolo record in formato JSON Lines (JSONL), forzando la scrittura su disco tramite `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il descrittore del file. In Phase 7, `store.merge_and_finalize()` recupera tutti i file `.jsonl` dalla directory temporanea, applica un ordinamento lessicografico basato sul `test_id` per preservare il determinismo dell'output, concatena i record in una singola collezione strutturata in formato JavaScript Object Notation (JSON) e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Riportando la soluzione iniziale con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente scartato dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell'audit trail senza produrre nessun errore visibile. Il design corrente elimina questa classe di bug per costruzione: ogni record scritto su disco è permanente, e il numero di record per test è illimitato. L'unico parametro che incide sulla dimensione dell'archivio è `RESPONSE_BODY_MAX_CHARS = 10 000`: la costante tronca il body di risposta di ogni singolo record per evitare che risposte molto grandi saturino il disco, lasciando il conteggio dei record invariato.

La proprietà di recuperabilità è un effetto collaterale dell'architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSONL-compatibile: l'evidenza parziale è recuperabile anche in assenza di un run completo.

5.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una superficie di rischio: `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. Delegare la redazione ai singoli connector presuppone che ogni implementazione, incluse quelle di terze parti, ricordi di oscurare i propri campi sensibili: l'adozione di logiche custom e frammentate

elimina la possibilità di definire garanzie uniformi, poiché ogni estensione opererebbe secondo criteri arbitrari e privi di coordinamento.

La responsabilità della redazione viene pertanto centralizzata nel punto di scrittura su disco, operando attraverso due flussi distinti in base alla natura dell'evidenza: gli artifact generati dai tool esterni e le transazioni HTTP prodotte dai test nativi. Gli artifact dei tool esterni passano per `EvidenceStore._sanitize_artifact()`, invocato automaticamente da `pin_artifact()` prima della scrittura su disco. Questo meccanismo solleva i connector dall'onere della pulizia dei dati e applica tre tecniche in sequenza sul dizionario `raw_output` grezzo :

1. **Key-pattern matching:** un'espressione regolare esamina ricorsivamente ogni chiave del dizionario rispetto a 11 pattern sensibili (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`, `access_token`). Il confronto richiede l'esatta corrispondenza della stringa e ignora le sottostringhe: questa scelta previene falsi positivi ed evita la redazione involontaria di campi legittimi (ad esempio, una chiave `author` non viene censurata pur contenendo la sequenza `auth`). Qualsiasi chiave con corrispondenza vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo.
2. **JWT value detection:** la regex `_SANITIZE_JWT_PATTERN` intercetta le stringhe che contengono la firma di un token JWT⁷, offuscando il valore indipendentemente dal nome della chiave che lo ospita; cattura quindi i token che appaiono come valori in chiavi con nomi arbitrari.
3. **Header prefix matching:** qualsiasi stringa che inizia con `"Bearer "`, `"Basic "`, `"Token "` o `"token "` viene redattata, intercettando i token che compaiono nei valori di header HTTP inclusi nei record di evidenza.

La sovrapposizione dei tre meccanismi copre scenari che nessuno dei tre individualmente intercetterebbe: un token JWT in una chiave `"result"` arbitraria supera il primo meccanismo (nome non riconosciuto) ma viene catturato dal secondo (struttura `base64url`); un header `Authorization` in un dict annidato viene catturato dal primo meccanismo (key-pattern matching ricorsivo). Spostando il presidio di sicurezza al confine dello storage, un connector di terze parti privo di controlli interni non compromette l'integrità dell'audit trail. Il sistema diventa intrinsecamente resiliente a implementazioni difettose, poiché la garanzia risiede nel punto di consolidamento e non nel punto di generazione del dato.

I tre meccanismi appena descritti si applicano esclusivamente agli artifact dei connector; le transazioni HTTP generate dai test nativi seguono invece un percorso se-

⁷La struttura standard di un token JWT si compone di tre parti distinte, denominate *header*, *payload* e *signature*, codificate in Base64URL e separate da un punto.

parato. La redazione non avviene a posteriori sul dizionario, ma interviene direttamente al livello del modello Pydantic: un `field_validator` applicato all'attributo `EvidenceRecord.request_headers` intercetta la creazione del record e sostituisce il valore dell'header `Authorization` con `[REDACTED]`. Questo approccio preventivo assicura che nessuna credenziale nativa raggiunga l'`EvidenceStore` in chiaro.

5.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I tre deliverable finali dell'assessment assolvono funzioni distinte e si rivolgono a interlocutori diversi. Il termine *Dual* si riferisce quindi al parallelismo tra la persistenza tecnica destinata alle macchine (`evidence.json`) e la presentazione operativa destinata all'analisi umana (`assessment_report.html`).

`evidence.json` è l'archivio forense. Contiene due categorie di record unite in un'unica struttura serializzata: le transazioni HTTP che i test nativi hanno marcato come evidenza di FAIL, con il body di risposta fino a 10.000 caratteri, e gli artifact dei tool esterni, ossia gli output JSON dei connector sanitizzati e pinned tramite `pin_artifact()`. Ogni record è autosufficiente per ricostruire la probe o l'analisi che ha rivelato la vulnerabilità. Il file affianca i metadati di intestazione (`generated_at_utc`, `record_count`) alla collezione lineare dei record, creando un formato JSON nativamente compatibile con l'ingestion da parte delle piattaforme di log aggregation.

`assessment_report.html` rappresenta il deliverable operativo, renderizzato da `report/renderer.py` tramite il motore Jinja2⁸. Il documento include le statistiche aggregate, la conformità normativa e il `TransactionSummary`: la cronologia completa delle transazioni per ciascun test, che attesta la *proof of coverage*, permettendo di verificare il perimetro interrogato dal tool e di fare triage senza aprire `evidence.json`. Il report è generato come artefatto autonomo, con stili Cascading Style Sheets (CSS) e script integrati *inline* per garantirne la consultazione offline senza dipendenze esterne. Le transazioni adottano inoltre tecniche di lazy loading per mantenere la reattività dell'interfaccia su dataset massivi.

`apiguard_report.json` fornisce la serializzazione strutturata dell'oggetto `ReportData`, il quale contiene tutti i risultati, le statistiche aggregate e i metadati della run. L'architettura garantisce una rigorosa identità strutturale tra questo documento (prodotto nativamente dalla CLI) e i dati JSON scaricabili in locale dall'interfaccia HTML del report. Questa sincronizzazione assicura che il `TransactionSummary` visionato dall'utente e i dati consumati dalle pipeline CI/CD o dai sistemi Security Information and Event Management (SIEM) coincidano esattamente, azzerando il rischio di incongruenze.

⁸Una libreria di templating per Python che consente la generazione dinamica di HyperText Markup Language (HTML) a partire da strutture dati tipizzate.

Capitolo 5 Implementazione

L'organizzazione visiva del report riflette la classificazione operata da `report/builder.py`: all'interno di ogni dominio di assurance, i verdetti dei test nativi precedono i risultati derivati dai tool esterni, sfruttando il campo `TestResult.source`. Questa separazione guida la lettura dell'analista, isolando le verifiche protocollari dirette dall'orchestrazione di strumenti di terze parti.

Capitolo 6

Validazione sperimentale e risultati

I Capitoli 4 e 5 hanno descritto come APIGuard Assurance è stato progettato e realizzato. Questo capitolo verifica empiricamente che quelle scelte architetturali si comportino nel modo dichiarato quando il tool viene eseguito contro un target reale. La validazione si articola in quattro livelli che formano una gerarchia di precondizioni: ciascuno è condizione di interpretabilità del successivo. La topologia dell'ambiente di test (Sezione 6.1) stabilisce le condizioni di osservazione. La qualità ingegneristica del codebase (Sezione 6.2) certifica che lo strumento di misura è affidabile. La copertura funzionale e l'idempotenza su due run indipendenti (Sezione 6.3) dimostrano che i risultati sono stabili e privi di effetti collaterali. Il profilo computazionale (Sezione 6.4) ne qualifica l'integrabilità operativa.

Tutti i dati quantitativi di questo capitolo sono il prodotto di verifiche condotte prima del rilascio della versione 0.1.0: misurazioni di sistema, esecuzioni ripetute dell'assessment e ispezioni statiche del codebase, ciascuna con un comando o una procedura riproducibile. Nessun risultato è stimato o interpolato. Un capitolo di risultati che non esplicita come sono stati prodotti i numeri che riporta è, dal punto di vista della riproducibilità, un capitolo di affermazioni; ogni dato che segue è corredato dalla procedura che lo ha generato.

6.1 Topologia del Testbed Sperimentale

La scelta dell'ambiente target non era vincolata a una specifica applicazione, ma a un insieme di requisiti derivati dalla matrice di copertura della suite: una specifica OpenAPI Specification (OAS) nativa non generata a posteriori, endpoint protetti da autenticazione reale, almeno due livelli di privilegi distinti, e comportamenti osservabili e stabili

attraverso run successivi senza modifiche al target. Forgejo 14.0.3, esposto attraverso Kong 3.9 in modalità DB-less, soddisfa questi requisiti. La scelta conferma inoltre in modo concreto l'agnosticismo applicativo discusso nella Sezione 4.1.1: il file `config.yaml` usato in questo assessment è strutturalmente identico a quello che si userebbe su qualsiasi altra API REST documentata, perché il tool non contiene nessun riferimento specifico a Forgejo.

6.1.1 Architettura del Testbed

Il testbed è dichiarato in un singolo file Docker Compose ed eseguito su host di sviluppo, senza hardware dedicato né virtualizzazione separata. I quattro servizi condividono il network `lab-net`: PostgreSQL 15-alpine come backend di persistenza per Forgejo, Forgejo 14 come applicazione target, Kong 3.9 DB-less come gateway che riceve tutto il traffico in ingresso, e un container `forgejo-setup` che configura gli account al primo avvio e poi termina. Kong raggiunge Forgejo attraverso un upstream dichiarato nel file di configurazione; il tool raggiunge Kong dall'host tramite le porte esposte. La Tabella 6.1 riporta la matrice delle versioni fissata durante l'audit.

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
Kong	3.9 DB-less	API Gateway: proxy, plugin, Admin API su porta 8001
PostgreSQL	15-alpine	Backend di persistenza Forgejo
nuclei	3.8.0	Tool esterno per Shadow API discovery (test ext.0.1)
nuclei-templates	10.4.3	Template pinned per riproducibilità delle scan
testssl.sh	3.2.3	Analisi TLS black-box (test ext.1.5.testssl)
sslyze	(libreria Python)	Analisi TLS via libreria (test ext.1.5.sslyze)

Tabella 6.1: Componenti del testbed e versioni al Milestone 1.

Kong espone due porte di proxy sull'host: la 8000 in HTTP semplice, usata dal test 1.5 per verificare il redirect verso Hypertext Transfer Protocol Secure (HTTPS), e la 8443 con certificato self-signed generato localmente, che costituisce l'endpoint primario per tutti gli altri test. L'Admin API è accessibile sulla porta 8001 e fornisce i dati di configurazione ai test `WHITE_BOX` dei Domini 3, 4 e 6. La configurazione è interamente

contenuta nel file `kong.yml`, montato in sola lettura nel container: Kong opera senza database, leggendo la configurazione dal file a ogni avvio. Questo è il pattern DB-less che la Sezione 4.3.2 identifica come prerequisito per il testing a visibilità piena sul piano di controllo del gateway, perché rende la configurazione ispezionabile e immutabile durante i run.

6.1.2 Configurazione Intenzionale per la Copertura dei Finding

Validare un tool di assessment su un ambiente completamente hardened produrrebbe una suite con tutti **PASS**: un risultato tautologico che non dimostra nulla sulla capacità del tool di rilevare violazioni reali. La configurazione del testbed prevede intenzionalmente alcune scelte non ottimali, che producono **FAIL** osservabili e attesi, e alcune configurazioni corrette, che producono **PASS** attesi. La combinazione rende il testbed un banco di prova funzionalmente completo: il tool deve saper distinguere i due casi.

Di conseguenza, tre scelte di configurazione producono **FAIL** osservabili e attesi nell'assessment:

1. Il plugin `rate-limiting` di Kong è commentato nel file `kong.yml`. Nessun limite di velocità è attivo sul servizio Forgejo. Il test 4.1, che invia richieste in loop verso un endpoint protetto fino a ricevere **HTTP 429**, non riceve mai quel codice e registra **FAIL** con 2 finding. Si tratta di una condizione comune in ambienti di staging dove il rate limiting viene disattivato per non interferire con i test applicativi, e quindi rappresenta uno scenario realistico.
2. La specifica OpenAPI di Forgejo dichiara un requisito di autenticazione globale a livello di documento, ma non tutti gli endpoint lo ereditano nella configurazione Kong del testbed: il gateway non applica autenticazione su molti di essi. Il test 1.1 sonda ogni endpoint della specifica senza credenziali e registra **BYPASSED** per ciascun endpoint che risponde **2xx**. Il verdetto è **FAIL** con 74 finding. Questo non è un falso positivo: il tool riporta correttamente la discrepanza tra ciò che la specifica dichiara e ciò che il gateway applica. Un endpoint marcato come autenticato nella specifica che risponde **2xx** a una richiesta priva di credenziali è una violazione reale, indipendentemente dall'intenzione del deployer.
3. Il certificato TLS self-signed presenta configurazioni non ottimali rilevabili dall'analisi statica del protocollo. I test `ext.1.5.sslyze` e `ext.1.5.testssl` registrano rispettivamente 1 e 3 finding sulla configurazione TLS.

Al contrario, il plugin `response-transformer` è attivo e inietta gli header di sicurezza HTTP su tutte le risposte del servizio Forgejo: **Strict-Transport-Security**, **Content-Security-Policy**, **X-Content-Type-Options** e **X-Frame-Options**. Il test 0.1, che veri-

fica la presenza di questi header, registra PASS. Il parametro `KONG_HEADERS=off` nella configurazione Kong rimuove l'header `Server: kong/version`, producendo PASS anche per il test 6.2 che verifica il fingerprinting del gateway. Il servizio `http-redirect-service` produce un redirect 301 con header `Location` corretto per qualsiasi richiesta HTTP sulla porta 8000, consentendo al test 1.5 di verificare il redirect HTTP→HTTPS e registrare PASS. La combinazione di finding attesi e PASS attesi rende il testbed un banco di prova funzionalmente completo: il tool deve saper rilevare il primo gruppo e non produrre falsi positivi nel secondo.

6.1.3 Struttura RBAC e Provisioning degli Utenti

Il container `forgejo-setup` crea tre account al primo avvio: `thesis-admin` (amministratore), `user-a` (utente con privilegi ordinari) e `user-b` (secondo utente non privilegiato). La struttura a tre livelli è il minimo necessario per la matrice di test: `thesis-admin` è il soggetto dei test di autenticazione che richiedono token con privilegi elevati; `user-a` è il soggetto del test 2.1 (RBAC enforcement), che tenta endpoint amministrativi con un token non-admin e attende 403; `user-b` è necessario per i test di isolamento tra oggetti di utenti distinti, che verificano che le risorse di `user-a` non siano accessibili con il token di `user-b`. I test 1.4 e 2.1 dipendono da token validi prodotti dal test 1.1, dipendenza dichiarata nel DAG come vincolo di Phase B e discussa nella Sezione 6.4.

Il provisioning è deterministico. Il container `forgejo-setup` usa l'interfaccia CLI di Forgejo con flag `--or-true`, così da non fallire se gli account esistono già da un avvio precedente. Questa scelta realizza concretamente la proprietà di riproducibilità discussa nella Sezione 4.1.4: il testbed raggiunge lo stesso stato iniziale indipendentemente da quante volte viene avviato, condizione necessaria perché i run successivi siano comparabili.

6.2 Release-Readiness e Integrità Architetture

Un tool che produce verdetti di sicurezza deve poter dimostrare che la propria implementazione è soggetta allo stesso rigore che applica al target. Un codebase con errori di tipo non rilevati, dipendenze con Common Vulnerabilities and Exposures (CVE) note, o simboli pubblici privi di contratto documentato produce output la cui affidabilità è difficilmente separabile dalla qualità del codice che lo genera. Prima di discutere cosa il tool ha trovato sul target, questa sezione stabilisce su quali basi ingegneristiche poggia quella analisi.

Le verifiche condotte coprono quattro aree: l'analisi statica del codebase tramite una suite di strumenti automatizzati (Sezione 6.2.1), la type safety a due strati che combina analisi statica e modelli Pydantic (Sezione 6.2.2), e gli indicatori di qualità dell'ingegneria

di produzione che riguardano documentazione, riproducibilità della build e installabilità (Sezione 6.2.3). I risultati sono riportati come prerequisito di credibilità, non come contributo autonomo.

6.2.1 Analisi Statica del Codebase

Su 90 file sorgente Python, la suite di analisi statica ha prodotto i seguenti esiti al momento dell'audit, riassunti nella Tabella 6.2.

Tool	Risultato	Dettaglio
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN: lint, import order, upgrade, security, annotation coverage
mypy	0 errori su 90 file	Modalità <code>strict = true</code> : <code>no-implicit-optional</code> , <code>disallow-untyped-defs</code> , <code>warn-return-any</code> e tutti i flag associati
bandit	0 finding severity medium+	3 Low (B404, S603×2 in <code>connectors/base.py</code>), tutti soppressi con <code># noqa: S603</code> documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence ≥80%: nessun simbolo pubblico inutilizzato. <code>execute</code> , <code>model_*</code> e <code>validator_*</code> esclusi per dispatching dinamico
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate

Tabella 6.2: Analisi statica della codebase APIGuard Assurance v0.1.0

Il sistema di type enforcement a due strati, discusso nella Sezione 6.2.2 del presente capitolo e distinto dalla dual test hierarchy della Sezione 5.3.1 del Capitolo 5,¹ formalizza un sistema con copertura complementare.

6.2.2 Dual-Layer Type Safety (D6.P3)

Il problema che questa proprietà risolve ha una struttura precisa. Su 90 file sorgente analizzati in modalità `strict`, il type checker non ha rilevato alcuna violazione: la verifica statica con mypy garantisce che il contratto tra i moduli, espresso tramite type hints, sia consistente lungo l'intera catena di dipendenze. Nessun punto di chiamata riceve un tipo incompatibile con quello dichiarato dal callee. Gli errori di interfaccia tra moduli vengono intercettati prima che il codice venga eseguito contro un target reale.

¹La Sezione 5.3.1 del Capitolo 5 tratta la dual test hierarchy (gerarchia delle classi di test); la proprietà D6.P3 di dual-layer type safety è un meccanismo distinto, il cui locus è in `pyproject.toml` e `src/core/models/`.

Mypy non copre però le system boundaries: il punto in cui dati esterni al codebase, un file `config.yaml` caricato da disco o il raw output di un connector, entrano nel sistema. Quel confine è presidiato da Pydantic v2. I `model_validator` e i `field_validator` dei modelli di configurazione applicano le invarianti al momento della costruzione dell'oggetto: un campo `timeout_seconds = None` con `enabled = True` nel `config.yaml` produce un `ConfigurationError` con il path del campo violato durante la Phase 1, non un crash non gestito durante l'esecuzione dei test. La classe di bug “typecheck verde, runtime crash” è strutturalmente eliminata.

I due strati sono indipendenti e complementari: ciascuno intercetta una categoria di errori che l'altro non raggiunge. Il costo mantenuto è prossimo allo zero per chi aggiunge un test: type hints e modelli Pydantic con `frozen=True` sono già vincolati dalle Hard Rules del progetto. La safety non dipende dalla disciplina individuale di chi contribuisce.

6.2.3 Indicatori di Qualità dell'Ingegneria di Produzione

Oltre all'analisi statica, l'audit ha verificato una serie di proprietà ingegneristiche che contribuiscono alla riproducibilità e all'affidabilità operativa del tool. Tre meritano menzione esplicita per la loro rilevanza rispetto alle garanzie di assurance che il tool promette.

Il primo è la copertura della documentazione interna: 292 simboli pubblici su 292 analizzati tramite scansione Abstract Syntax Tree (AST) presentano una docstring di modulo o di classe (100%). In un tool estensibile per design, la documentazione dei contratti pubblici non è decorativa: è il meccanismo attraverso cui un nuovo test può capire cosa può chiamare e con quali aspettative di comportamento.

Il secondo è la riproducibilità della build. Due esecuzioni consecutive di `hatch build --target wheel` producono un artefatto con SHA-256 identico (451 261 byte). Una wheel non deterministica introduce una categoria di variabilità nel deployment che è metodologicamente inaccettabile per un tool che promette idempotenza sui propri risultati.

Il terzo è il cold install: in un ambiente virtuale Python pulito, senza alcuna dipendenza preinstallata, l'esecuzione di `pip install apiguard_assurance-0.1.0-py3-none-any.whl` seguita da `apiguard version` restituisce 0.1.0. Il tool è self-contained e non dipende da pacchetti installati nell'ambiente dell'esecutore.

Il verdetto complessivo dell'audit, sintetizzato nella Tabella 6.3, è: pronto per la difesa di tesi e per il deployment in ambiente chiuso su target con OAS documentata. Gli unici due item deferiti, il file `LICENSE` e il `CHANGELOG.md`, riguardano esclusivamente la distribuzione pubblica su PyPI e non hanno impatto sul comportamento operativo del tool.

Aspetto	Stato	Riferimento
Correttezza funzionale	✓	Analisi statica 100% verde su 90 file (verifiche 1–5)
Integrità architetturale	✓	18 <code>test_id</code> unici; DAG aciclico; gerarchia eccezioni 11 classi; dipendenze monodirezionali (verifiche 19–23)
Copertura documentazione	✓	100% docstring su 292 simboli pubblici; 265/265 Pydantic <code>Field</code> description (verifica 65)
Packaging e distribuzione	✓	Wheel 451 261 byte, SHA-256 identico su 2 build consecutive (verifica 67)
Cold install	✓	<code>pip install wheel</code> in fresh venv $\rightarrow$ <code>apiguard version</code> $\rightarrow$ 0.1.0 (verifica 64)
Resilienza di produzione	✓	Signal handling garantisce teardown Phase 6 su <code>KeyboardInterrupt</code> (verifica 47)
Item deferiti	2	<code>LICENSE</code> e <code>CHANGELOG.md</code> : solo per distribuzione PyPI pubblica, nessun impatto operativo

Tabella 6.3: Verdetto release-readiness al Milestone 1.

6.3 Copertura Funzionale e Idempotenza dell’Assessment

L’idempotenza è una proprietà delicata che richiede di essere attestata empiricamente. Test che creano risorse sul target, le interrogano, e poi le eliminano introducono dipendenze di stato che, se mal gestite, producono risultati diversi al secondo run: la prima esecuzione modifica il target, la seconda trova uno stato diverso. Che il sistema produca risultati identici su due run indipendenti è la dimostrazione empirica che la pipeline non lascia effetti collaterali osservabili, non un’affermazione di principio.

I due run sono stati eseguiti in sequenza nello stesso giorno contro lo stesso testbed, senza alcun intervento manuale tra l’uno e l’altro. La Tabella 6.4 riporta i Key Performance Indicator (KPI) aggregati; la Tabella 6.5 dettaglia lo status e il conteggio dei finding per ciascuno dei 18 test attivi.

KPI	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
<code>pass_rate_pct</code>	56.2%	56.2%	identico
<code>assessment_duration_seconds</code>	286.04 s	286.96 s	+0.32%
Exit code / label	1 / FAIL	1 / FAIL	identico

Tabella 6.4: KPI di idempotenza su due run indipendenti.

Test ID	Dominio	Tipo	Status	Finding
0.1	D0 — Discovery	Nativo	FAIL	16
0.2	D0 — Discovery	Nativo	FAIL	1
0.3	D0 — Discovery	Nativo	SKIP	0
1.1	D1 — Auth	Nativo	FAIL	74
1.4	D1 — Auth	Nativo	PASS	0
1.5	D1 — Auth	Nativo	PASS	0
1.6	D1 — Auth	Nativo	SKIP	0
2.1	D2 — Authorization	Nativo	PASS	0
3.3	D3 — Data Integrity	Nativo	PASS	0
4.1	D4 — Availability	Nativo	FAIL	2
4.2	D4 — Availability	Nativo	PASS	0
4.3	D4 — Availability	Nativo	PASS	0
6.2	D6 — Hardening	Nativo	PASS	0
6.4	D6 — Hardening	Nativo	PASS	0
7.2	D7 — Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	D0 — Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1 — Auth	Esterno	FAIL	1
ext.1.5.testssl	D1 — Auth	Esterno	FAIL	3
Totale				98

Tabella 6.5: Status e finding count per test.

6.3.1 Analisi dei Risultati per Dominio

La distribuzione dei finding non è uniforme. Il test 1.1, che sonda ogni endpoint della specifica OpenAPI di Forgejo in assenza di autenticazione, produce da solo 74 dei 98 finding totali: il 75.5% dell'intero conteggio. Questo non è un artefatto di un test mal calibrato. Forgejo espone una superficie documentata di oltre cento endpoint, e la configurazione Kong del testbed non applica autenticazione a molti di essi per design sperimentale (come discusso nella Sezione 6.1). Ciascun endpoint che risponde **2xx** a una richiesta non autenticata è un finding distinto nell'oracle a tre esiti della Safe HTTP Probing Policy (Sezione 5.3.5, D4.P7): la cardinalità riflette la superficie reale del target, non un'amplificazione artificiale del contatore.

I due SKIP meritano una lettura separata. Il test 0.3 verifica che gli endpoint marcati **deprecated: true** nella specifica restituiscano **410 Gone** o portino un header **Sunset** valido (RFC 8594). La specifica OpenAPI di Forgejo 14.0.3 non dichiara nessun endpoint con questo marcatore: lo scope è legittimamente vuoto, e il test restituisce **SKIP** con motivo documentato. Il test 1.6 verifica la configurazione del session store tramite l'Admin API di Kong; nella configurazione del testbed, Kong non espone un session store

gestito esternamente, e la condizione di applicabilità non è soddisfatta. In entrambi i casi, SKIP non è un fallimento silenzioso: è la risposta semanticamente corretta all'assenza di condizioni di applicabilità, distinta dall'ERROR che indicherebbe un malfunzionamento della pipeline stessa.

I tre FAIL del Dominio 1 prodotti dai tool esterni riguardano la configurazione TLS. Il certificato self-signed del testbed presenta configurazioni subottimali rilevabili dall'analisi statica del protocollo: `ext.1.5.sslyze` rileva 1 finding, `ext.1.5.testssl` ne rileva 3. Questi risultati sono attesi e coerenti con l'uso di un certificato generato localmente per ambiente di laboratorio.

6.3.2 Idempotenza e Validazione del Teardown

Il delta sul wall-clock tra i due run è di 0.92 secondi su una durata media di circa 286 secondi, pari allo 0.32% della durata totale. Questo margine rientra interamente nel rumore delle latenze di rete HTTP verso il target e non è distinguibile dalla variabilità attesa di un sistema operativo in esecuzione su hardware condiviso. I contatori di finding, gli status per-test, il codice di uscita e il `pass_rate_pct` sono invece byte-identici: non vi è alcuna variazione nella parte deterministica del risultato.

L'idempotenza ha una preconditione concreta: il teardown deve restituire il target allo stato iniziale dopo ogni run, senza lasciare risorse residue che alterino i risultati dell'esecuzione successiva. La verifica live condotta dopo ciascuno dei due run ha confermato che la Phase 6 ha drenato il registro LIFO con 4 risorse transitorie e 0 fallimenti su entrambe le esecuzioni. La Tabella 6.6 riporta lo stato delle risorse osservabili sul target prima del primo run e dopo ciascuno dei due.

Risorsa sul target	Prima run 1	Dopo run 1	Dopo run 2	Verdetto
Token Forgejo thesis-admin contenenti "apiguard"	0	0	0	✓
Repository Forgejo user-a con prefisso "apiguard"	0	0	0	✓

Tabella 6.6: Verifica live del teardown post-run.

Zero risorse leakate su due run è la dimostrazione empirica che le proprietà discusse nella Sezione 5.3.8 (Best-Effort Teardown LIFO, D4.P6) e nella Sezione 4.1.4 (Riproducibilità, D3.P4) si comportano nel modo dichiarato in presenza di stato reale. La correttezza

teorica della logica di teardown trova il proprio banco di prova nello stato osservabile che produce: zero risorse residue è la misura diretta. Queste tabelle sono quella verifica.

6.4 Footprint Computazionale e Benchmarking del DAG

Il profilo di risorse di un tool di assessment è un criterio operativo primario. Un tool che satura la CPU, esaurisce la memoria o produce un output non ingestibile non è integrabile in una pipeline CI/CD senza hardware dedicato. I dati di questa sezione rispondono a una domanda precisa: su quale classe di macchina e in quale regime di esecuzione APIGuard Assurance è operativamente sostenibile?

La Tabella 6.7 riporta le metriche di sistema misurate sui due run dell'audit, ottenute tramite `/usr/bin/time -v` applicato al processo `apiguard run`.

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
CPU utilization (media)	26%	(simile)
Peak resident set size	287 MB	295 MB
Voluntary context switches	145.356	(simile)
Involuntary context switches	20.685	(simile)

Tabella 6.7: Metriche di sistema sui due run.

6.4.1 Regime di Esecuzione: IO-Bound, non CPU-Bound

Il dato più informativo nella tabella non è il wall-clock. È la relazione tra user time, system time e tempo totale. Su 290s di esecuzione reale, il processo ha consumato circa 35s di tempo CPU in user space e 41s in kernel space: la somma è 76s, il 26% del wall-clock. I restanti 214s, il 74% del tempo totale, il processo li ha trascorsi in attesa: attesa delle risposte HTTP dal target, attesa del completamento dei processi esterni (nuclei, `testssl.sh`), attesa dell'I/O su disco per la scrittura dell'evidence store. Ne emerge un'inferenza architetturale diretta: il collo di bottiglia è la latenza di rete verso il target, non l'interprete Python né il DAG scheduler. Qualsiasi refactoring del codice Python lascerebbe il wall-clock sostanzialmente invariato. Ciò che sposta effettivamente il wall-clock è la parallelizzazione dei test che non hanno dipendenze reciproche, traiettoria discussa nella Sezione 7.1 come sviluppo futuro di Milestone 2. In assenza

di parallelizzazione, il profilo attuale è il lower bound dell'architettura sequenziale, non un'inefficienza correggibile con refactoring locale.

Il profilo di memoria è compatibile con l'integrazione CI/CD su runner standard. 287–295 MB di resident set size si collocano ampiamente entro i limiti delle configurazioni GitHub Actions e GitLab Continuous Integration (CI) di default (tipicamente 7 GB per runner condivisi), senza richiedere runner dedicati né configurazioni di memoria particolari. Il footprint su disco è 4.5 MB: `apiguard_report.json` da 2.26 MB, `assessment_report.html` da 2.0 MB e `evidence.json` da 254 KB. I tre file sono autosufficienti e non richiedono dipendenze esterne per essere consultati.

6.4.2 Topologia del DAG e Implicazioni sull'Ordinamento dell'Esecuzione

L'esecuzione sequenziale di 18 test in due fasi non è la struttura più semplice realizzabile. Sarebbe stato possibile eseguire tutti i test in un singolo batch senza DAG: meno codice, zero overhead di scheduling. Il problema che questa scelta avrebbe lasciato aperto è la correttezza dei risultati in presenza di dipendenze tra test.

Il test 1.4 (revoca del token) richiede un token valido prodotto da un login HTTP riuscito. Il test 2.1 (RBAC enforcement) richiede lo stesso token per tentare endpoint amministrativi con credenziali di utente ordinario. Entrambi dipendono dal fatto che il test 1.1 abbia già interrogato la superficie di attacco e che l'autenticazione abbia prodotto un token nel `TestContext`. Eseguire 1.4 o 2.1 in assenza di questo prerequisito non produce un `ERROR` in fase di bootstrap: produce risultati non deterministici, ovvero output che dipendono dall'ordine di esecuzione e non dall'effettivo comportamento del target.

La topologia del DAG all'audit di Milestone 1, riportata nella Figura concettuale della Tabella 6.8, risolve il problema alla radice: 16 test senza dipendenze formano la Phase A ed eseguono per primi; 1.4 e 2.1, che dichiarano `depends_on = ["1.1"]`, vengono schedulati in Phase B e non possono mai essere avviati prima che il test 1.1 abbia prodotto un risultato.

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1, 0.2, 0.3, 1.1, 1.5, 1.6, 3.3, 4.1, 4.2, 4.3, 6.2, 6.4, 7.2, ext.0.1.nuclei, ext.1.5.testssl, ext.1.5.sslyze
Phase B (<code>depends_on = ["1.1"]</code>)	2 test	1.4, 2.1

Tabella 6.8: Topologia del DAG a Milestone 1.

Due considerazioni emergono da questa topologia. La prima è che il DAG è sparso: su 18 nodi, solo 2 hanno dipendenze dichiarate, e queste convergono su un unico predecessore. L'overhead di scheduling introdotto dal `DAGScheduler` è quindi trascurabile rispetto al tempo di esecuzione dei test; la complessità del componente si giustifica non sul carico attuale ma sulla scalabilità futura, quando dipendenze tra domini distinti renderanno il grafo più denso.

La seconda considerazione riguarda la relazione tra topologia del DAG e footprint di memoria. Con 16 test in esecuzione sequenziale nella Phase A, il `EvidenceStore` mantiene aperto un file `.jsonl` per volta: il picco di memoria non è determinato dal numero di test paralleli ma dalla dimensione del singolo audit trail. I 287–295 MB misurati rappresentano il profilo reale di questa esecuzione, non una stima conservativa. La transizione verso l'esecuzione parallela dei test all'interno della stessa fase, prevista in Milestone 2, richiederà una rivalutazione di questo profilo: più file `.jsonl` aperti concorrentemente comportano un footprint proporzionalmente maggiore, da misurare prima di fissare i requisiti di memoria per deployment CI/CD.

Sintesi della Validazione

I quattro livelli di analisi si stratificano in una gerarchia di precondizioni: ciascuno è condizione di interpretabilità del successivo. Il testbed stabilisce le condizioni di osservazione. La release-readiness certifica che lo strumento di misura è affidabile. La copertura funzionale e l'idempotenza dimostrano che i risultati prodotti sono stabili e non contaminati da effetti collaterali. Il profilo computazionale ne qualifica l'integrabilità operativa. Rimuovere uno qualsiasi di questi livelli non impoverisce la validazione: la invalida.

Il risultato aggregato è leggibile in termini precisi. Su un target reale con una OAS nativa, `APIGuard Assurance v0.1.0` ha eseguito 18 test distribuiti su 7 domini di sicurezza, prodotto 98 finding in 9 categorie distinte, restituito risultati byte-identici su due run indipendenti, drenato tutte le risorse transitorie senza residui, e completato l'intero assessment in circa 4 minuti e 50 secondi su hardware non dedicato. Nessuno di questi numeri è stimato: tutti tracciano a un punto di verifica nell'audit.

La validazione empirica misura comportamenti osservabili e non può pronunciarsi sui limiti del paradigma che il tool implementa. Un assessment corretto su una specifica errata produce verdetti formalmente validi ma sostanzialmente privi di significato. Un tool che astrae il gateway per garantire portabilità multi-vendor rinuncia, per costruzione, ai controlli vendor-specifici privi di un equivalente generalizzabile. Queste non sono anomalie dell'implementazione: sono tensioni strutturali del contratto che `APIGuard`

Capitolo 6 Validazione sperimentale e risultati

Assurance stringe con il proprio dominio applicativo. La loro analisi è l'oggetto del Capitolo 7.

Capitolo 7

Discussione e Sviluppi Futuri

Il Capitolo 6 ha risposto a una domanda circoscritta: APIGuard Assurance produce i risultati dichiarati su un target reale? I dati confermano. Rimane aperta una domanda di portata diversa: entro quali confini strutturali quella risposta vale, e come il sistema si inserisce in contesti d'uso più ampi di quello sperimentale?

Questo capitolo esamina quei confini in tre direzioni. La prima è critica: i limiti strutturali del paradigma contract-driven, ciascuno radicato in una proprietà architettureale specifica e non eliminabile con un refactoring locale, vengono discussi senza attenuarli (Sezione 7.1). La seconda è applicativa: le condizioni concrete in cui il tool è integrabile in pipeline CI/CD industriali e i pattern di deployment che ne massimizzano il valore (Sezione 7.2). La terza è prospettica: le traiettorie di sviluppo distinte in estensioni operative che seguono direttamente dall'architettura attuale, e direzioni di ricerca che pongono problemi aperti e richiedono validazione sperimentale indipendente (Sezione 7.3).

7.1 Analisi Critica e Limiti del Paradigma Contract-Driven

Ogni proprietà architettureale che garantisce un vantaggio specifico impone, per costruzione, una classe di situazioni in cui quel vantaggio diventa vincolo. I limiti discussi in questa sezione non sono anomalie correggibili per via implementativa: sono conseguenze dirette di scelte deliberate, e come tali richiedono di essere nominati con precisione piuttosto che attenuati.

7.1.1 Il Paradosso del Ground Truth (D3.P2)

Come stabilito nella Sezione 4.1.2, la specifica OpenAPI è l'unico oracolo formale della pipeline: delimita la superficie di attacco, guida la costruzione dei probe e fornisce il contratto rispetto al quale le risposte vengono interpretate. Il vantaggio è la precisione semantica: dove un fuzzer cieco produce HTTP 400 a causa di payload strutturalmente invalidi, APIGuard Assurance costruisce richieste conformi al contratto e sposta l'osservazione dalla sintassi alla semantica del controllo di sicurezza. Come discusso nella Sezione 4.1.2, questo è precisamente il *combinatorial wall* che il paradigma contract-driven abbatte.

Il prezzo è la dipendenza dalla qualità di ciò che si assume come oracolo. Una specifica che non riflette endpoint aggiunti dopo l'ultima pubblicazione restringe la superficie di attacco osservabile senza che il sistema possa rilevare l'incoerenza: l'assenza di un endpoint dall'assessment è indistinguibile dalla sua assenza dalla specifica. Una specifica costruita per escludere endpoint sensibili produce un assessment formalmente corretto su una superficie artificialmente ristretta. In entrambi i casi il tool non dispone di un meccanismo per distinguere l'assenza di vulnerabilità dall'assenza di copertura — lo *specification drift* tra documentazione e implementazione reale è una delle cause principali di copertura incompleta nel testing contract-driven, indipendentemente dallo strumento adottato. Qualsiasi approccio che utilizzi una specifica formale come oracolo eredita la stessa dipendenza: la correttezza dell'assessment è necessariamente condizionale alla correttezza del contratto. La traiettoria di ricerca che affronta strutturalmente questo paradosso, l'inferenza automatica della specifica dall'analisi del traffico reale tramite tecniche di Machine Learning (ML), è discussa nella Sezione 7.3 come direzione aperta, non come soluzione disponibile.

In termini operativi, la mitigazione praticabile è procedurale: l'operatore che esegue l'assessment è responsabile di verificare che la specifica fornita sia aggiornata e completa prima di interpretare i risultati. Il tool può segnalare questa dipendenza, ed è quello che fa producendo un SKIP con motivazione esplicita quando uno scope è vuoto; ma non può validare autonomamente che lo scope rifletta fedelmente il sistema reale.

7.1.2 La Tensione dell'Astrazione (D1.P6)

La Sezione 4.2.4 ha introdotto `BaseGatewayAdapter` come interfaccia astratta verso il piano di configurazione del gateway: i test `WHITE_BOX` interrogano il gateway attraverso questa interfaccia, e aggiungere il supporto per un nuovo prodotto richiede una nuova implementazione concreta senza toccare i test che la utilizzano. Il vantaggio è la portabilità; il vincolo conseguente è che l'interfaccia può esporre solo ciò che è concettualmente generalizzabile a tutti i gateway che la implementano.

Un esempio rende la tensione concreta. Kong supporta l'ordinamento esplicito dei plugin tramite il campo `ordering` della sua Admin API: un plugin di autenticazione eseguito dopo un plugin di trasformazione del payload riceve dati già alterati, con potenziali implicazioni sulla sicurezza. Verificare questa condizione richiede metadati Kong-specifici privi di equivalente strutturale in altri gateway; un test che li utilizzasse dovrebbe dipendere da `KongGatewayAdapter` direttamente, rompendo l'agnosticismo. Il design attuale non percorre questa strada.

La tensione non è necessariamente irrisolvibile. Una direzione di sviluppo percorribile è l'estensione della gerarchia dell'adapter: `BaseGatewayAdapter` rimane l'interfaccia comune con i metodi generalizzabili, ma classi intermedie come `BaseKongAdapter` o `BaseAWSAdapter` potrebbero esporre metodi vendor-specifici per i test che li richiedono, senza impattare i test che operano sul livello comune. Questa struttura a tre livelli rifletterebbe la stessa logica della gerarchia dei connector descritta nella Sezione 5.4.1. Un deployment che usa esclusivamente Kong potrebbe adottare test scritti contro `BaseKongAdapter`; un deployment multi-vendor userebbe solo i test del livello comune. La ricerca necessaria riguarda la specifica del contratto di compatibilità tra i livelli, non la fattibilità tecnica dell'approccio.

7.1.3 La Dipendenza dall'Admin API (D4.P5)

La degradazione controllata descritta nella Sezione 5.5.2 garantisce che l'assenza della Admin API non blocchi la pipeline, ma non elimina le conseguenze sulla copertura. In ambienti cloud managed, come Amazon API Gateway, Azure API Management (APIM) o Google Cloud Endpoints, il piano di configurazione non è esposto tramite un'Admin API interrogabile direttamente: è accessibile solo attraverso le API del cloud provider, con modelli di autenticazione specifici per piattaforma. Configurare `gateway_adapter` per questi ambienti richiede un adapter dedicato per ciascun provider, attualmente non implementato.

La copertura si riduce ai domini `BLACK_BOX` e `GREY_BOX`: D0, D1, D2, D3, D4 parzialmente, D7. Il Dominio 6 (Configuration & Hardening) e le verifiche `WHITE_BOX` di D4 restano escluse. La riduzione non è silenziosa: ogni `SKIP` riporta nel report la motivazione esplicita, rendendo la copertura effettiva immediatamente leggibile.

La prassi industriale che affronta questo limite è l'accordo di accesso privilegiato: nell'ambito di un assessment formale, il team di sicurezza ottiene accesso read-only alle API di gestione del cloud provider per la durata dell'analisi, tipicamente con credenziali temporanee e scope limitato alle risorse del progetto. Questo modello presuppone un accordo esplicito con il provider o con il team di governance interna, e garantisce che la lettura della configurazione avvenga senza possibilità di modifica. Con quell'accesso disponibile, `gateway_adapter` nel `config.yaml` viene popolato con le credenziali

e l'endpoint dell'adapter specifico per il provider, e la copertura si estende ai domini `WHITE_BOX` senza modifiche al codice. La condizione è che quell'adapter esista: la sua implementazione per i cloud provider principali è una delle estensioni operative discusse nella Sezione 7.3.1.

7.2 Applicabilità Industriale e Integrazione DevSecOps

Le proprietà di riproducibilità e degradazione controllata discusse nei Capitoli 4 e 5 non sono solo garanzie interne al tool: definiscono le condizioni concrete in cui APIGuard Assurance è integrabile come componente di una pipeline CI/CD industriale. Questa sezione descrive quelle condizioni: la semantica del canale di comunicazione con la pipeline, i due profili di esecuzione disponibili e la variabilità di copertura nei diversi contesti organizzativi.

7.2.1 Semantic Exit Codes come Canale di Comunicazione con la Pipeline (D6.P1)

Il contratto tra APIGuard Assurance e la pipeline che lo ospita è espresso da quattro valori interi. Quando il processo termina, l'exit code riflette lo stato aggregato del `ResultSet` calcolato da `report/builder.py`: 0 indica che nessun test ha prodotto `FAIL` o `ERROR`; 1 indica che almeno un test ha prodotto `FAIL`; 2 indica che nessun test ha prodotto `FAIL` ma almeno uno ha prodotto `ERROR`; 10 indica un errore infrastrutturale bloccante nelle fasi di startup (Fase 1, 2 o 4), ovvero il processo non ha mai raggiunto l'esecuzione dei test. La priorità tra i codici è ordinata: `FAIL` prevale su `ERROR`, quindi un assessment con entrambe le condizioni restituisce 1.

La separazione tra 1 e 10 è il punto architetturealmente rilevante. Da una pipeline automatizzata, "almeno un test ha trovato una violazione" e "il tool non si è avviato" sono condizioni con diagnosi e azioni correttive completamente diverse; trattarle con lo stesso valore scarica sull'operatore l'onere di leggere i log per distinguerle. Con exit code semanticamente distinti, un alert su 10 indica un problema infrastrutturale nel deployment della pipeline, mentre un alert su 1 indica che l'assessment è stato completato e ha rilevato almeno una garanzia violata.

L'integrazione in sistemi di CI/CD come GitHub Actions o GitLab CI¹ non richiede wrapper aggiuntivi: il check sull'exit code è il meccanismo nativo. Un passo che esegue `apiguard run --config config.yaml` blocca la pipeline se il tool restituisce exit 1:

¹GitHub Actions e GitLab CI sono piattaforme di integrazione continua che eseguono pipeline di build, test e deployment automatizzato al variare del codice sorgente. Entrambe trattano qualsiasi exit code diverso da zero come fallimento del passo, bloccando l'avanzamento della pipeline.

in un workflow di merge request, questo impedisce l'integrazione del codice finché le garanzie di sicurezza non sono soddisfatte. Il report prodotto rimane disponibile come artefatto scaricabile per l'analisi post-hoc; il gate decisionale è il codice di uscita.

La granularità attuale è binaria rispetto all'esito: o l'assessment è pulito o non lo è. Una traiettoria di sviluppo naturale è rendere il predicato di blocco configurabile per livello di priorità, così che una violazione di priorità P0 su un endpoint di autenticazione blocchi la pipeline mentre una violazione P3 su una configurazione subottimale produca un avviso senza interrompere il rilascio. Questa direzione è discussa nella Sezione 7.3.1 come estensione operativa.

7.2.2 Quality Gate a Due Velocità: Fail-Fast e Assessment Completo (D4.P8)

L'assessment completo descritto nel Capitolo 6 ha richiesto circa cinque minuti su un target con 18 test attivi. In un gate su una merge request, dove l'obiettivo è verificare rapidamente che le modifiche non abbiano introdotto regressioni di sicurezza perimetrale, attendere il completamento dell'intero assessment è un costo sproporzionato rispetto alla decisione da prendere: se un test P0 rileva una violazione critica, il blocco della pipeline è già giustificato prima che i test successivi vengano eseguiti.

Il parametro `execution.fail_fast` nel `config.yaml` introduce questa seconda dimensione di controllo, distinta dagli exit code ma complementare ad essi: mentre la Sezione 7.2.1 descrive cosa il tool comunica al termine dell'esecuzione, `fail_fast` descrive se e quando il tool interrompe l'esecuzione prima del completamento naturale. Con il valore di default `false`, l'engine esegue tutti i test attivi e produce un report completo. Con `fail_fast: true`, l'engine interrompe la Phase 5 alla prima violazione confermata su un test di priorità P0: i test rimanenti non vengono eseguiti, il `ResultSet` contiene solo i risultati prodotti fino a quel punto, e il processo termina con exit 1. La condizione di interruzione include anche `ERROR` sui test P0, perché una verifica perimetrale incompleta per malfunzionamento del tool è operativamente equivalente a una violazione confermata: in entrambi i casi il controllo non garantisce ciò che deve garantire.

I due profili si adattano a scenari d'uso distinti. La modalità fail-fast è appropriata per i gate sulle merge request, dove la velocità di risposta ha valore: all'interruzione alla prima violazione P0, il wall-clock si riduce da cinque minuti a pochi secondi nel caso peggiore. La modalità completa è appropriata per assessment periodici schedulati su rami stabili, dove ogni dominio deve produrre evidenza per informare le decisioni di remediation. La combinazione con `execution.min_priority` nel `config.yaml` aggiunge un'ulteriore dimensione: limitare l'esecuzione ai soli test P0 in modalità fail-fast produce un gate veloce sui controlli perimetrali critici; estendere a tutti i livelli di priorità in

modalità completa produce l'assessment di profondità. Il medesimo tool, lo stesso file di configurazione di base, due comportamenti configurabili senza modifiche al codice.

7.2.3 Variabilità di Privilegio nei Deployment Industriali

Il tool adatta la propria copertura alle precondizioni disponibili senza che l'operatore scelga esplicitamente quale "modalità" attivare: la variazione è nell'insieme di credenziali fornite nel `config.yaml` e nel campo `gateway_adapter`, e la degradazione verso livelli di copertura inferiori avviene in modo automatico e documentato, come stabilito nella Sezione 4.3.2. Nei deployment industriali, questa proprietà si traduce in tre scenari concreti che riflettono le diverse condizioni di accesso delle organizzazioni che adottano il tool.

In contesti di integrazione continua su rami di sviluppo, i token di autenticazione per i ruoli configurati sono disponibili come segreti di pipeline, e la Admin API del gateway è accessibile dall'interno della rete di staging. Tutti e tre i livelli del box gradient sono raggiungibili, e l'assessment copre l'intera matrice dei domini `BLACK_BOX`, `GREY_BOX` e `WHITE_BOX`.

In contesti di verifica su ambienti di pre-produzione condivisi tra più team, le credenziali con privilegi amministrativi potrebbero non essere distribuite a tutti gli esecutori della pipeline per ragioni di governance interna. Il tool opera sui livelli `BLACK_BOX` e `GREY_BOX`: i test `WHITE_BOX` producono `SKIP` con motivazione esplicita, e la riduzione di copertura è documentata nel report.

In contesti di red team o penetration test esterno, solo l'accesso di rete al target è garantito. Il tool opera in modalità `BLACK_BOX`: i test `GREY_BOX` e `WHITE_BOX` producono `SKIP`, e l'assessment si concentra sui controlli perimetrali verificabili da un attaccante senza credenziali. In questo scenario, la copertura ridotta non è una limitazione da correggere: è il dato che misura la postura perimetrale del sistema così come la vedrebbe un attaccante esterno.

7.3 Sviluppi Futuri

Le traiettorie di sviluppo si articolano su due scale temporali con natura diversa. Le estensioni operative sono conseguenze dirette dell'architettura corrente: connector progettati e non ancora implementati, domini predisposti nella tassonomia e non ancora coperti, componenti di pipeline che seguono direttamente dai meccanismi già validati. Non richiedono scelte di design nuove. Le traiettorie di ricerca pongono invece problemi aperti che richiedono analisi formale o validazione sperimentale indipendente prima

dell'implementazione, e in alcuni casi costituiscono oggetti di studio autonomi. La distinzione non è di importanza ma di natura: confonderle produrrebbe impegni di consegna su lavoro che richiede ricerca.

7.3.1 Estensioni Operative

Le estensioni elencate in questa sezione seguono direttamente dall'architettura corrente: i punti di estensione sono già predisposti, i contratti tra i componenti sono già definiti. La mancata implementazione nella fase attuale riflette scelte di priorità, non vincoli architetturali. Le estensioni si raggruppano in tre aree.

Copertura dei test e connectors. Il Dominio 5 (Observability) è l'unico non coperto da nessun test nella fase attuale, come dichiarato nella Sezione 4.3.1. I test 5.1 (audit logging) e 5.2 (alert real-time su eventi anomali) sono già specificati nella metodologia; la loro implementazione richiede l'estensione del testbed con un log aggregator e un sistema di alerting nel Docker Compose, dipendenze di infrastruttura che si mantengono architetturealmente isolate dal core del tool.

Tra i connector Categoria A non ancora implementati, il `JwtToolConnector` sblocca la copertura del ciclo di vita delle credenziali nel Dominio 1: i test 1.2 (validazione strutturale del JWT) e 1.3 (expiry check), pianificati come `depends_on = ["1.1"]`, sono schedulati in una fase successiva del DAG e non possono avviarsi finché test 1.1 non ha prodotto un token valido nel `TestContext`. La struttura del DAG è già predisposta per accogliere questa Phase C senza interventi sull'ordinamento topologico. Analogamente, il Dominio 2 (Authorization) nella fase attuale copre solo il test 2.1; i test 2.2–2.5, pianificati con il connector OFFAT per la generazione dinamica di probe BOLA/Insecure Direct Object Reference (IDOR) derivati dalla specifica OpenAPI, completerebbero la copertura del controllo accessi a livello di oggetto. Estendere il Dominio 3 con `Schemathesis` per l'injection testing, il Dominio 6 con `ext.6.3.socket` per HTTP request smuggling, e il Dominio 7 con connector Vegeta e Interactsh per race condition e SSRF blind completano il quadro dei connector Cat A pianificati.

Due di questi meritano una nota tecnica specifica. Il test 7.3 (Time-of-Check to Time-of-Use (TOCTOU)) richiede richieste simultanee con sincronizzazione sub-millisecondo: il Global Interpreter Lock (GIL) di Python rende questo requisito inaffidabile, e il `VegetaConnector` delega la generazione del carico a Vegeta, scritto in Go con goroutine native, ottenendo la finestra di concorrenza necessaria. Il connector è riutilizzabile dal test 4.1 per il load testing del rate limiting, in coerenza con il principio di separazione dati/valutazione discusso nella Sezione 5.4.1. L'SSRF blind (test `ext.7.4.interactsh`) richiede invece un oracle fuori banda: `InteractshConnector` registra un URL controllato su un server interactsh remoto, lo inietta nel probe e verifica il callback ricevuto.

L'evidenza non è nella risposta HTTP al probe, ma nella richiesta in uscita dal server verso l'indirizzo controllato.

Integrazione della pipeline. Due estensioni riguardano il canale di comunicazione con la pipeline CI/CD, già descritto nella Sezione 7.2.1. La prima è la granularità degli exit code per livello di priorità, che permetterebbe di distinguere violazioni P0 (bloccanti in qualsiasi scenario) da violazioni P2/P3 (segnalabili come avvertimenti senza bloccare il rilascio). La seconda è l'implementazione degli adapter per i cloud provider principali (Amazon API Gateway, Azure APIM), che consentirebbe la copertura `WHITE_BOX` nei deployment cloud managed discussi nella Sezione 7.1.3.

Ottimizzazioni del motore. La struttura a batch del DAG è già parallelizzabile per costruzione, come discusso nella Sezione 5.2.2: i test all'interno di uno stesso batch non hanno dipendenze reciproche. Introdurre un `ThreadPoolExecutor` nella Phase 5 ridurrebbe il wall-clock proporzionalmente al numero di test eseguibili in parallelo, con un effetto stimabile dalla distribuzione osservata nella Sezione 6.4. La ricerca necessaria riguarda i problemi di thread-safety su `EvidenceStore` e `TestContext`, discussi nella Sezione 7.3.2.

7.3.2 Traiettorie di Ricerca Aperta

Le direzioni descritte in questa sezione non sono estensioni implementative: sono problemi che l'architettura attuale abilita o porta alla luce, ma che richiedono analisi formale, validazione sperimentale indipendente o scelte di design nuove prima di poter essere affrontati concretamente.

Parallelizzazione intra-batch e thread-safety. L'estensione della parallelizzazione al livello intra-batch, anticipata nella Sezione 7.3.1, richiede la risoluzione di due problemi di thread-safety che non hanno soluzioni banali. Il primo riguarda l'`EvidenceStore`: il metodo `begin_test()` apre un file in append e lo mantiene aperto per la durata del test; con più test concorrenti che chiamano `log_transaction()`, le scritture devono essere serializzate senza introdurre un lock globale che annulli il vantaggio della parallelizzazione. Il secondo riguarda il `TestContext`: il canale dei token è condiviso tra tutti i test, e chiamate concorrenti a `acquire_tokens()` potrebbero produrre login multipli per lo stesso ruolo, violando la proprietà di idempotenza discussa nella Sezione 5.3.3. La strada percorribile, per entrambi i problemi, è la separazione dei contesti per batch anziché per run, con un `TestContext` locale a ogni batch e un meccanismo di merge controllato alla fine di ciascuno. L'analisi formale della correttezza di questo schema è il prerequisito all'implementazione.

APIGuard come piattaforma modulare. L'architettura attuale è costruita attorno al paradigma REST/OpenAPI: i test nativi derivano la superficie di attacco dalla specifica OAS, i connector integrano tool pensati per quel protocollo, e il modello di evidenza riflette la semantica di richiesta/risposta HTTP. Esistono tuttavia classi di API che questo modello non raggiunge: endpoint GraphQL, servizi gRPC, sistemi event-driven basati su AsyncAPI. La traiettoria di ricerca è trattare APIGuard Assurance non come un tool monoprodotto ma come una piattaforma di assurance estendibile, dove il core — engine, DAG scheduler, evidence store, report — rimane invariato, e i moduli di protocollo vengono aggiunti come plugin indipendenti. Un modulo `apiguard-graphql` contribuirebbe i propri test nativi e connector per GraphQL rispettando i contratti di `BaseTest` e `BaseConnector`, senza modificare il core. Le sfide di ricerca riguardano la specifica formale del contratto di compatibilità tra moduli e core al variare delle versioni, la gestione della sicurezza dei moduli contribuiti da terze parti, e la definizione di un modello di evidenza comune abbastanza astratto da rappresentare semantiche di protocollo diverse.

Drift detection tra specifica e comportamento osservato. Il paradosso del Ground Truth discusso nella Sezione 7.1.1 — la specifica può essere obsoleta o incompleta senza che il tool possa rilevarlo — apre una traiettoria di ricerca che non richiede traffico passivo né inferenza statistica. Durante l'esecuzione, il tool raccoglie già osservazioni sistematiche sul comportamento del target: status code restituiti, header presenti o assenti, schemi di risposta effettivi. Confrontare formalmente queste osservazioni con le asserzioni della specifica OpenAPI potrebbe rivelare endpoint non documentati che rispondono, comportamenti difformi dalla specifica dichiarata, e silenziosamente produrre un indicatore di confidenza sulla completezza dell'oracolo usato. Questa direzione, diversa dall'inferenza di una specifica sintetica dal traffico, usa i dati che il tool già produce come input per una valutazione della qualità dell'oracolo stesso. Le sfide aperte riguardano la definizione formale di "difformità significativa", la distinzione tra deviazioni intenzionali e involontarie, e l'integrazione dell'indicatore nel report senza trasformarlo in un ulteriore vettore di falsi positivi.

Capitolo 8

Conclusioni

Tre tensioni hanno orientato le scelte di questo lavoro: l'incapacità degli approcci tradizionali di rilevare vulnerabilità semantiche nelle API REST, l'assenza di strumenti che combinino agnosticismo applicativo con testing guidato dal contratto formale, e la non confrontabilità dei risultati di un assessment tra esecuzioni successive. Il framework sviluppato risponde a ciascuna con meccanismi architetturali verificabili empiricamente. Questo capitolo ripercorre quella risposta, ne valuta la portata effettiva e colloca il contributo nel contesto più ampio della sicurezza come proprietà misurabile del processo di sviluppo.

8.1 Sintesi del Lavoro e Risoluzione del Problema Iniziale

La cecità semantica degli approcci tradizionali, analizzata nel Capitolo 3 come distanza tra ciò che uno scanner osserva e ciò che una vulnerabilità logica richiede di capire, appartiene alla struttura del paradigma: questa barriera persiste indipendentemente dal volume di payload generati, perché un fuzzer che ignora il contratto API affronta il combinatorial wall prima ancora di raggiungere la logica applicativa, con la quota di richieste che violano i tipi e i vincoli enumerati nello schema che viene respinta con `HTTP 400` dal layer di validazione. Usare la specifica OpenAPI come oracolo formale, formalizzato nella Sezione 4.1.2, abbatte questa barriera spostando il punto di osservazione dalla sintassi alla semantica: ogni richiesta è costruita nel rispetto dei vincoli contrattuali, ogni risposta è valutata rispetto a ciò che il contratto prevede per quella specifica operazione. Ne diventa raggiungibile la classe di vulnerabilità invisibili al fuzzing cieco: le violazioni di autorizzazione orizzontale (BOLA), l'elusione della revoca dei token e gli abusi di endpoint che accettano URL controllabili dal chiamante, tutte categorie che presuppongono richieste sintatticamente corrette e semanticamente illecite.

L'agnosticismo applicativo, realizzato attraverso la derivazione a runtime di tutta la conoscenza del target dalla specifica OpenAPI e dal `config.yaml` come stabilito nella Sezione 4.1.1, estende questo approccio oltre il singolo caso specifico. Un tool che incorpora conoscenza hardcoded del target, path, strutture dati e comportamenti attesi, è applicabile a quel target e soltanto a quello: sostituita l'applicazione, va riscritto. Un tool che deriva quella conoscenza dalla specifica formale è applicabile a qualsiasi API REST documentata senza modifiche al codice, con il vincolo che una specifica valida sia disponibile; questo vincolo, dichiarato come limite strutturale nella Sezione 7.1, appartiene al paradigma contract-driven nel suo complesso. La profondità dell'analisi si preserva perché il contratto governa la costruzione delle richieste e la valutazione delle risposte con la stessa precisione indipendentemente dal target. L'approccio ibrido risultante occupa lo spazio che i tool vendor-specific e i fuzzer generici lasciano scoperto congiuntamente: i primi sono precisi ma vincolati a un sistema specifico, i secondi sono portabili ma privi di conoscenza contrattuale.

La terza tensione, la non confrontabilità tra esecuzioni successive che esclude il penetration testing tradizionale dall'integrazione come quality gate, trova risposta nella riproducibilità deterministica discussa nella Sezione 4.1.4 e validata empiricamente nella Sezione 6.3. Un framework che produce risultati identici su esecuzioni successive dello stesso target non afferma la riproducibilità come proprietà attesa: la dimostra come invariante misurabile, condizione necessaria perché un assessment automatizzato possa assumere il ruolo di quality gate in una pipeline di sviluppo continuativo.

8.2 Contributi Scientifici e Ingegneristici

I contributi di questo lavoro si articolano su due piani distinti che si rinforzano senza sovrapporsi: la formalizzazione metodologica, che produce strumenti analitici riutilizzabili indipendentemente dall'implementazione specifica, e la realizzazione ingegneristica, che dimostra la concreta praticabilità di quella metodologia con le garanzie di qualità richieste da un artefatto industriale.

Sul piano metodologico, tre contributi si stratificano in ordine crescente di specificità implementativa. La tassonomia a otto domini di assurance, descritta nella Sezione 4.3.1, definisce cosa misurare: organizza la superficie di sicurezza di un'API esposta tramite gateway in categorie semanticamente distinte, ciascuna con le proprie precondizioni di accesso e i propri criteri di valutazione, applicabile a qualsiasi assessment strutturato indipendentemente dagli strumenti impiegati. Il box gradient, formalizzato nella Sezione 4.3.2, aggiunge la dimensione del privilegio e definisce come accedere alla misura: la classificazione Black/Grey/White Box viene derivata dalle precondizioni concrete che ogni garanzia impone al tester, producendo un mapping analitico tra il livello di visibilità disponibile e la classe di controlli raggiungibile. Le trentanove proprietà architettura-

li distribuite su sette domini, raccolte nella Tabella 4.1 e trattate nei Capitoli 4 e 5, definiscono come costruire il sistema che misura: un vocabolario di design che cattura le decisioni architetturali con le loro motivazioni e i costi accettati, riutilizzabile come modello per sistemi analoghi.

Sul piano ingegneristico, il design di un assessment engine deterministico basato su DAG, descritto nella Sezione 4.2.2, costituisce un contributo architettuale autonomo: la struttura a batch con risoluzione topologica garantisce la correttezza semantica dell'ordine di esecuzione indipendentemente dal numero e dalla natura dei test, una proprietà che uno scheduler a priorità statica non raggiunge perché non possiede la rappresentazione delle dipendenze tra test. La codebase che implementa questo design è stata validata empiricamente contro un target reale: su Forgejo 14.0.3 esposto tramite Kong 3.9 DB-less, due run indipendenti hanno prodotto risultati identici per tutti i contatori aggregati (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, delta wall-clock dello 0,32%), come verificato nella Sezione 6.3. La qualità del codice è verificabile per analisi statica: 0 errori mypy in strict mode su 90 file sorgente, come documentato nella Sezione 6.2.2; un tool che si propone come strumento di assurance applica alla propria implementazione lo stesso rigore che applica ai target che valuta.

8.3 Considerazioni Finali sull'Assurance Continuo

La differenza tra il penetration testing tradizionale e l'automated security assurance è epistemica, non di scala. Il penetration testing produce un referto: un'osservazione puntuale condotta in un momento specifico, da un operatore con un insieme specifico di conoscenze e tecniche; rieseguire il test in condizioni nominalmente identiche produce output confrontabili solo per convenzione, perché quella variabilità è incorporata nel metodo. L'automated security assurance produce una garanzia: lo stesso assessment, rieseguito con la stessa configurazione sullo stesso target, produce risultati deterministicamente identici, verificabili da chiunque senza ricostruire il processo mentale di chi l'ha condotto. La riproducibilità dimostrata nella Sezione 6.3 non è una proprietà accessoria del sistema: è la condizione che trasforma un'osservazione in un'affermazione verificabile da terzi.

APIGuard Assurance è la dimostrazione che la sicurezza delle API può essere trattata come una proprietà dell'ingegneria del software: verificabile per analisi statica e per esecuzione, tracciabile attraverso un audit trail forense, integrabile nel ciclo di sviluppo tramite exit code semantici senza richiedere intervento manuale a ogni release. Applicare alle API lo stesso rigore che si applica al codice, dalla verifica dei tipi alla regressione automatizzata fino ai quality gate di pipeline, non è un'aspirazione teorica: è ciò che un framework deterministico e config-driven rende operativamente praticabile. Quando l'assurance è continuo, la sicurezza smette di essere un attributo certificato una volta al-

Capitolo 8 Conclusioni

l'anno e diventa una proprietà del processo stesso, contestata a ogni commit e confermata a ogni deploy.

Bibliografia